



THE STABILITY GAP AS A TRAJECTORY PROBLEM: A BENT DESCENT PATH, ITS OVERSHOOT, AND A SINGLE GATE

Bachelor's Thesis

Arthur Ernst Henrik Reuss, s5582253, a.e.h.reuss@student.rug.nl,

Supervisors: Guido van de Ven

Abstract: Replay-based continual learning is usually judged by the accuracy it reaches *after* each task. Continual evaluation exposes a failure this view hides: the *stability gap*, a sharp but transient drop in past-task accuracy at the start of every new task. The gap persists even when the optimiser descends the exact joint loss over all tasks, so its cause lies in the optimisation trajectory rather than the objective. This thesis explains the gap on two levels. Learning both tasks has a fixed price: the loss on past tasks must rise to its level at the joint optimum. An envelope-theorem argument shows that a reference path exists along which this rise is monotone. Steepest descent after an abrupt task switch leaves that path: it bows out of the old task's basin and returns from outside, a deterministic arc that survives exact gradients (level one, the trajectory problem). Real optimisers also overshoot whatever path they follow: an imbalanced first step and gradient noise from a finite buffer push the iterate beyond it (level two, the overshoot problem). Written in a unified update rule, every replay method counters these problems with the same tool, a *gate* on the new-task gradient, while the replay gradient restores at full strength. We build and compare the two pure gate designs. The λ -*curriculum* is a scalar feedback gate: it ramps the new-task loss weight up from zero, so the optimiser tracks the moving valley floor of the reference path. *Asymmetric preconditioned experience replay* is a directional feedforward gate: it filters the new-task gradient through past-task curvature, passing it where the past task is flat and braking it where the past task is steep. On rotated MNIST the two levels separate cleanly. Removing the overshoot drivers leaves the arc; placing the optimiser on the reference path leaves the overshoot; doing both removes the gap while the fixed price remains. The curriculum needs momentum for accuracy, not for the gap: momentum acts as a variance reducer that closes the residual gap and as an accelerator that restores plasticity. The directional gate lowers the gap monotonically as its filter strengthens, at zero plasticity cost, but it fails through blind spots of its sampled curvature metric and its protection unravels under momentum. Where a gate attenuates thus decides how it composes with momentum: at a matched memory budget and no momentum the directional gate is strictly better, but momentum, required for accuracy, re-inflates its per-step filter, and the scalar gate takes over in the regime that ships. On CIFAR-10 with a ResNet-18 the curriculum cuts gap depth several-fold below vanilla experience replay at matched final accuracy and at negligible compute cost, under two normalisation schemes and on corruptions it was never tuned on.

Contents

1	Introduction	3
2	Background and Related Work	4
2.1	Continual Learning and Experience Replay	4
2.2	The Stability Gap	4
2.3	One Update, Many Methods	5
2.4	Low-Loss Paths and Mode Connectivity	5
3	The Stability Gap as a Trajectory Problem	5
3.1	The Reference Path and the Tracking Bound	5
3.2	The Two-Level Gap: Bowing and Overshoot	6
3.3	Two Gates: Attenuating the New-Task Gradient	6

4	Research Questions	7
5	Methods	8
5.1	Experimental Setup	8
5.2	Experiments	9
6	Results	11
6.1	The Two Levels, Separated (RQ1)	11
6.2	The Scalar Feedback Gate (RQ2)	13
6.3	The Directional Feedforward Gate (RQ3)	15
6.4	Reading the Momentum Crosses Together	15
6.5	Generalisation to CIFAR-10 (RQ4)	17
6.6	Summary of Findings	17
7	Discussion	19
7.1	Why the gap needs two levels	19
7.2	Two gates, two ways to divide the work	19
7.3	Limitations	20
7.4	Outlook	21
	Statement on the Use of AI Tools	21
A	Implementation Details	24
A.1	Datasets	24
A.2	Models and Training	24
A.3	Experience replay	24
A.4	Asymmetric PER	25
A.5	Natural Continual Learning	25
B	Metric Definitions	26
C	Complete Results	27
C.1	rot-MNIST: Decomposition (D-series)	27
C.2	rot-MNIST: λ -Curriculum (C-series)	28
C.3	rot-MNIST: Buffer-Fidelity Diagnostics	28
C.4	rot-MNIST: Symmetric vs. Asymmetric Preconditioning	28
C.5	CIFAR-10: Headline Block	29
C.6	CIFAR-10: Generalisation Block	29
C.7	CIFAR-10: Momentum Cross and Compute Cost	30
C.8	Long-Sequence rot-MNIST (L-series)	31
C.9	Supplementary Figures	31
D	Detailed Step-by-Step Derivation of the Trajectory Bound	34

1 Introduction

Deep neural networks learn powerful representations from static, independent, and identically distributed data, but a vulnerability appears as soon as training becomes sequential: the network overwrites previously acquired knowledge to accommodate new data. This phenomenon, catastrophic forgetting [18, 21], is a major hurdle in machine learning. Retraining from scratch solves the issue, but it is too expensive for systems that need to adapt continuously. Continual learning (CL) studies how to learn from a sequence of tasks without that cost, and its core difficulty is the stability–plasticity dilemma: the model must stay plastic enough to learn new distributions while staying stable enough to preserve old ones [19].

Replay, the family this thesis studies, stores a small buffer of past examples and trains on them alongside the new data. Replay methods reduce long-term forgetting well, but until recently they were evaluated only at task boundaries, which hides what happens within a task. Lange et al. [16] evaluated the network after every update and identified the *stability gap*: a severe but transient collapse in past-task accuracy that occurs immediately when a new task begins, followed by a slow recovery. This matters for two reasons. A deployed system is queried throughout training, so a model caught mid-transition commits errors its final checkpoint never would. And because the model eventually recovers, past knowledge is disrupted rather than lost, which means the disruption itself is the target for repair.

The discovery raised a natural question: would a better approximation of the joint loss over all tasks remove the gap? Hess et al. [11] showed that the gap persists even under incremental joint training, where the model has access to all data from all tasks at every step. A perfect objective still produces transient forgetting. The problem therefore lies in *how* the optimisation proceeds, and this thesis locates it on two levels: the descent path itself is bent, and real optimisers overshoot whatever path they follow.

Two levels. Figure 1.1 visualises the transition as a switch from a learned task (Task A) to a new task (Task B), a conceptual reduction of a high-dimensional surface using the loss-landscape lens of Kao et al. [14]. *Level one* is geometric. Starting at the converged minimum A , steepest descent on the joint loss follows the green curve: because the steepest direction differs from the direction toward the joint minimum $A \cap B$ whenever curvature is anisotropic, the trajectory bows out of Task A’s basin and returns from outside [14], so the Task-A loss rises above its final level and comes back. This arc is deterministic, it appears with exact gradients and vanishing step size, and its signature is a hanging tail. This increase is an artifact of the specific trajectory rather than

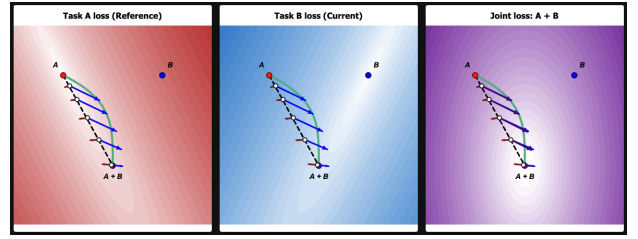


Figure 1.1: 2D optimisation landscape at a task transition. Single-task losses Task A (red) and Task B (blue) are minimised at A and B ; the joint loss (purple) is minimised at $A \cap B$. The steepest-descent trajectory (green curve) illustrates the trajectory problem (Section 3.2), bowing outside A ’s basin into regions of higher loss. Conversely, the dashed black chord sketches the monotone reference path of Section 3.1, which stays inside the basin. Overshoot drivers (Section 3.2) further push the iterate off these paths.

an unavoidable cost. Reaching the joint optimum requires only that the past-task loss monotonically rise to its higher joint value. Section 3.1 constructs a reference path along which the loss does exactly this, sketched as the dashed chord in Figure 1.1, and the mode-connectivity literature confirms that such low-loss routes exist in practice [6, 20]. Everything the accuracy curve does below its settled level and back is therefore avoidable in principle. *Level two* is that real optimisers overshoot whatever path they follow. At the boundary the objective switches in one step from the replay loss to the joint loss, so the target teleports from θ_A^* to θ_{joint}^* while the parameters stand still, leaving the iterate maximally imbalanced, replay gradient near zero, new-task gradient largest [16]. A finite step on that imbalanced push carries the iterate far out of the old basin, and finite-buffer gradient noise [1] widens and prolongs the excursion. This overshoot acts on the bowed path and the reference alike. Section 3 develops both levels.

One tool: gate the new-task gradient. Sequential replay methods share a single update, a weighted combination of the current-task and replay gradients [22]. Section 3.3 generalises this: every replay method applies a *gate* to the new-task gradient and lets the replay gradient restore at full strength, $d = G g_{\text{cur}} + g_{\text{rep}}$. The gate’s *domain* can be scalar (attenuate the whole push) or directional (attenuate per direction), and its *control* feedback (react to a live signal) or feedforward (commit to a schedule or precomputed filter). This thesis builds and compares the two pure designs. The first is the λ -**curriculum**, a scalar gate. Instead of switching abruptly to the joint loss, it ramps the new-task weight from 0 to 1 over N steps,

$$L_\lambda(t) = L_{\text{replay}} + \lambda(t) L_{\text{current}}, \quad (1.1)$$

so the minimiser of the objective moves along a continuous path from θ_A^* to θ_{joint}^* and the model tracks a moving valley floor. Its adaptive variant sets λ from the ratio of the replay and new-task gradient norms, a live damage signal, which makes it a scalar *feedback* gate. The second is **asymmetric preconditioned experience replay** (asymmetric PER), a directional *feedforward* gate. It filters the new-task gradient through the curvature of the replay loss,

$$d = \delta (F_{\text{rep}} + \delta I)^{-1} g_{\text{cur}} + g_{\text{rep}}, \quad (1.2)$$

where F_{rep} is the replay Fisher information and δ sets the filter strength. The push passes at full gain where the past task is flat and is braked where the past task is steep, while the restoring replay gradient stays untouched.

Contributions. This thesis makes four contributions.

1. **A two-level account of the stability gap.** The gap separates into a deterministic arc of the descent path and overshoot of the followed path, with the permanent stability–plasticity cost split off by a monotone reference path (Section 3). Controlled experiments confirm the separation: removing the overshoot drivers leaves the arc, fixing the path leaves the overshoot, and doing both removes the gap.
2. **A unified gate view of replay methods,** which places existing methods and the two designs of this thesis in one two-axis taxonomy and turns the taxonomy’s predicted failure modes into testable hypotheses (Section 3.3).
3. **The λ -curriculum,** a scalar feedback gate with an $O(1/N)$ tracking guarantee, an adaptive schedule, a $\lambda_{\min}$ plasticity floor, and a characterisation of its interplay with momentum.
4. **Asymmetric PER,** a directional feedforward gate, with a damping sweep that maps its stability–plasticity trade-off and identifies its failure modes: blind spots of the sampled curvature metric and the unravelling of per-step protection under momentum.

2 Background and Related Work

2.1 Continual Learning and Experience Replay

Methods against catastrophic forgetting fall into a few families: replay, parameter and functional regularisation, optimisation-based methods, context-dependent processing, and template-based classification [25].

This thesis focuses on replay, specifically its simplest formulation: Experience Replay (ER) [3]. At each step, ER draws a replay mini-batch from a buffer of stored past examples, combines it with the current-task mini-batch, and takes one optimiser step on the summed loss.

A standard approach to managing this episodic memory is reservoir sampling [26], which ensures that every sample seen so far occupies a buffer slot with equal probability. Under this setup, the buffer remains empty during the first task, meaning ER reduces to plain SGD; consequently, performance gaps and forgetting metrics are evaluated from the second task onward. This foundational simplicity is why ER remains highly competitive—a small episodic memory often outperforms purpose-built methods and scales to longer sequences [3]. Furthermore, because the ER update consists of exactly two ingredients, it provides a remarkably clean substrate for isolating and studying interventions at task boundaries.

However, a finite buffer is inherently an imperfect estimator of the past-task distribution. Aljundi et al. [1] frame buffer selection as approximating the feasibility region of the full past data, with reservoir sampling serving as one specific policy within that framework. Consequently, the replay gradient derived from a small buffer will differ in both direction and magnitude from the true gradient of all past data.

2.2 The Stability Gap

Lange et al. [16] identified the stability gap using continual evaluation, measuring past-task accuracy after every update rather than only at task boundaries. In this view a new task begins with substantial but transient forgetting that the model subsequently recovers from, and they quantify it with worst-case metrics, mainly the minimum past-task accuracy reached during training, rather than with end-of-task averages alone. The effect is not confined to one method: they observe it across experience replay, constraint-based replay, knowledge distillation, and parameter regularisation. It also persists well beyond the standard benchmarks, appearing in joint incremental learning of homogeneous tasks [13] and in continual pre-training of large language models [7], and several mitigations have been proposed [9, 23].

Two findings in this literature anchor the account of Section 3. First, Lange et al. [16] trace the initial drop to a gradient imbalance at the boundary: the new-task gradient dominates the first updates, so the optimiser trades stability for plasticity precisely when the past task is most exposed. Second, the gap is a property of the trajectory rather than the objective. Hess et al. [11] show it persists under training on the exact joint loss with full access to all past data, and Kao et al. [14] demonstrate the geometric mechanism analytically: steepest descent from an old-task mini-

num leaves the old basin even with perfect joint gradients, because the steepest direction and the direction to the joint minimum disagree. A perfect objective therefore fails to prevent the gap, and the descent geometry itself must be addressed.

2.3 One Update, Many Methods

A second class of replay-adjacent methods constrains the update at the task boundary using information about past tasks. GEM stores past-task gradients and projects the new-task gradient onto the nearest direction that leaves every stored loss unchanged or lower [17]. A-GEM replaces the per-task constraints with a single averaged past-task gradient, reducing the projection to one cheap operation [2]. EWC adds a quadratic penalty around the past optimum weighted by a curvature estimate [15], and natural-gradient approaches precondition the update with past-task curvature so that steps avoid directions the old task depends on [14].

On the surface these methods look unrelated, but Sodhani et al. [22] show the replay-based ones share a single update. Every step combines two gradients, the current task’s and the replay buffer’s, under two scalar weights,

$$\theta \leftarrow \theta - \eta (\alpha_1 \nabla L_{\text{current}} + \alpha_2 \nabla L_{\text{replay}}), \quad (2.1)$$

so a method reduces to a rule for choosing α_1 and α_2 . Plain ER fixes both to one. GEM and A-GEM hold $\alpha_1 = 1$ and raise the replay weight α_2 only when the new-task and stored gradients conflict, by just enough to cancel the interference; A-GEM’s projection is therefore the addition of scaled losses. MEGA [8] likewise holds $\alpha_1 = 1$ but sets the replay weight from a loss ratio, $\alpha_2 = L_{\text{replay}}/L_{\text{current}}$, leaning on memory once the new task is already fit.

The same reading applies to the λ -curriculum of this thesis. It fixes $\alpha_2 \equiv 1$ and turns the other knob, ramping the current-task weight $\alpha_1 = \lambda(t)$ up from zero. It belongs to the same family as A-GEM and MEGA; what distinguishes it is which weight it moves and what signal drives the weight. The adaptive variant (Section 5.2.2) is the mirror image of MEGA: MEGA raises the *replay* weight from a *loss* ratio to recruit memory once the new task is learned, while the adaptive curriculum holds the *new task* back with a *gradient-norm* ratio, letting it in only as the replay gradient reasserts itself. Section 3.3 generalises Equation (2.1) from scalar weights to operators and organises all of these methods, together with asymmetric PER, along two design axes.

2.4 Low-Loss Paths and Mode Connectivity

Mode connectivity gives empirical backing to the *monotone reference path* introduced in Section 3: the

route from the old-task solution θ_A^* to the joint solution θ_{joint}^* along which the past-task loss only ever rises toward its final value, rather than first climbing above it and coming back down as the bowed descent does. For such a path to be usable, the two solutions must first be joined by a corridor of low loss at all, and this is what the mode-connectivity literature reports. Garipov et al. [6] and Draxler et al. [4] showed that independently trained minima of a deep network are joined by curved paths of near-constant low loss, and Frankle et al. [5] sharpened this to straight paths for networks that share early training. For continual learning specifically, Mirzadeh et al. [20] found that the sequentially trained and the multitask solutions often lie in the same basin, connected by a straight low-loss line. A benign, low-loss route from θ_A^* to θ_{joint}^* should therefore exist in trained networks. Where it does, the stability gap reflects the optimiser straying off that route, and an intervention succeeds by keeping it there.

3 The Stability Gap as a Trajectory Problem

This section reframes the stability gap as a two-level trajectory problem. Learning a new task incurs a permanent stability–plasticity cost; the gap is the transient excess above it. This excess has two sources: a geometric bow in the ideal descent path, and overshoot that real training dynamics add on top. We formalise the split by constructing a monotone reference path, then show that a single design choice, how the update gates the new-task gradient, controls both sources. Two contrasting gates instantiate it: an adaptive scalar curriculum and an asymmetric directional filter.

3.1 The Reference Path and the Tracking Bound

Learning a second task inevitably moves the model. In a quadratic approximation with task losses L_A and L_B and Hessians H_A and H_B , the displacement $d = \theta_{\text{joint}}^* - \theta_A^*$ from the old optimum to the joint minimum permanently raises the replay loss by roughly $\frac{1}{2} d^\top H_A d$. Every method pays this price; it is distinct from the stability gap.

Yet this rise can be strictly monotone. Consider the curriculum objective $L_\lambda = L_{\text{replay}} + \lambda L_{\text{current}}$. As λ grows from 0 to 1, the minimiser traces a *reference path* $\Theta^*(\lambda)$ from θ_A^* to θ_{joint}^* . Under local positive-definiteness, an envelope-theorem argument (Appendix D) guarantees that the replay loss climbs directly to its final level with no excursion above it:

$$\frac{d}{d\lambda} L_{\text{replay}}(\Theta^*(\lambda)) \geq 0 \quad \text{for all } \lambda \in [0, 1]. \quad (3.1)$$

This guarantee is local: it holds only while the combined Hessian $H_A + \lambda H_B$ stays positive-definite along the tracked branch of minima. The condition is automatic near θ_A^* and persists until the new task’s negative curvature overpowers the old task’s; on a deep non-convex backbone it therefore serves as motivation rather than a guarantee.

Figure 3.1 shows this: as λ grows, the loss contours deform continuously from the old-task basin to the joint basin and the minimiser slides along the connected valley floor. Because the replay loss climbs directly to its final level, any transient excess *above* it, the stability gap, is a preventable deviation from the reference path.

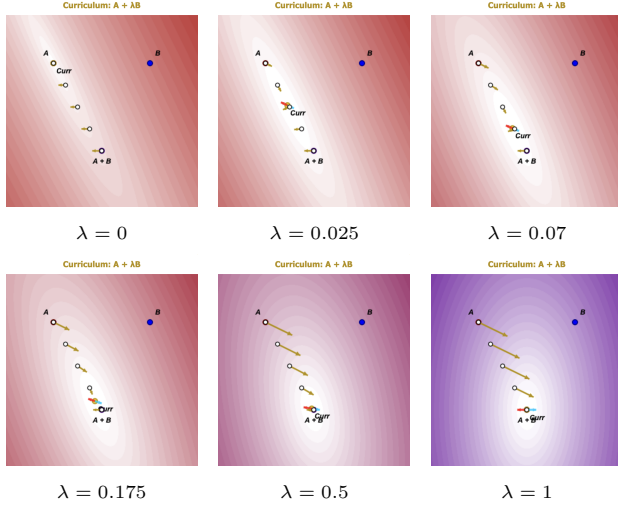


Figure 3.1: Evolution of the curriculum loss $L_\lambda = A + \lambda B$. As λ increases, the contours morph continuously from the old-task basin around A to the joint basin around $A+B$. The current minimiser (*Curr*) slides along the moving valley floor, forming the reference path $\Theta^*(\lambda)$. At the minimiser the replay (red) and current-task (blue) contributions cancel by the first-order condition $\nabla L_{\text{replay}} + \lambda \nabla L_{\text{current}} = 0$; their resultant (golden) vanishes there and, away from the minimiser, points down the valley, so a tracking model rides the moving floor instead of teleporting across the landscape.

A real iterate cannot sit on this path exactly, but it can stay close to it. Introducing the new task gradually over an N -step ramp ($\lambda \approx 1/N$) keeps the iterate tethered to the moving valley floor $\Theta^*(\lambda(t))$ rather than letting it leap across the landscape; a gradient-flow analysis bounds the resulting lag by $O(1/N)$ (Appendix D). Because the monotone path fixes the endpoint at the permanent level, this caps the highest transient replay loss of the whole transition,

$$\max_t L_{\text{replay}}(\theta(t)) \leq L_{\text{replay}}(\theta_{\text{joint}}^*) + O\left(\frac{1}{N}\right), \quad (3.2)$$

the first term the permanent price, the second a transient that shrinks with ramp length. A slow ramp is

precisely what the λ -curriculum of this thesis implements.

3.2 The Two-Level Gap: Bowing and Overshoot

Real trajectories deviate from the reference path on two distinct levels.

Level One (The Bow): Even under idealised conditions (exact gradients, vanishing step size), the steepest-descent direction of the abruptly assembled joint loss misaligns with the direction toward the joint minimum whenever the loss curves more sharply in some directions than in others. The trajectory bows outward, leaving the old task’s basin before returning [14]. This deterministic out-and-back arc forms the structural core of the gap (Figure 1.1, green curve).

Level Two (Overshoot): Real training dynamics inflate this arc. At a sudden task boundary, the minimum teleports from θ_A^* to θ_{joint}^* , leaving the iterate exposed to a large new-task gradient while the replay gradient is near zero [16]. The acute driver is this imbalance met by a finite step size: the first discrete steps ride the unopposed push far out of the old basin, creating the sharp first-step drop. A chronic driver, estimator variance from finite buffer sampling (Section 2.1), injects noise into the restoring gradient at every step; it slows the recovery tail and admits sudden drops long after the boundary.

The two levels shape the gap’s two dimensions: depth records the overshoot, whether the acute boundary spike or a late noise-driven drop, while the hanging tail is the Level-One arc plus the slow, noisy recovery the chronic driver leaves behind. This yields the falsifiable core, formalised as RQ1 in Section 4: removing the overshoot drivers should flatten the drop and leave the arc, placing the optimiser on the reference path should remove the arc and leave the overshoot, and doing both should close the gap while the permanent cost of Section 3.1 remains.

3.3 Two Gates: Attenuating the New-Task Gradient

Equation (2.1) shows that replay methods differ mainly in how they weight the replay and current-task gradients. The replay gradient is the only restoring force pulling the iterate back to the reference path, so a protective method must attenuate the new-task push, the source of the bow and overshoot, while leaving the restoring gradient intact. Generalising the current-task weight to an operator G yields a one-sided gate:

$$d = G g_{\text{cur}} + g_{\text{rep}}. \quad (3.3)$$

The curriculum of Section 3.1 realises this gate along one axis, *time*: it introduces the new task slowly, so the push is never large enough at once to knock the iterate off the reference path. A complementary axis

is *direction*. At any single step only part of the push is harmful, because the past task is flat in most directions and steeply curved in a few, and only motion along the steep directions raises the replay loss. Preconditioning the push by past-task curvature exploits this, passing it at full gain where the past task is flat and shrinking it where the past task is steep, so retention costs little plasticity. This natural-gradient idea underlies Natural Continual Learning [14] and, in a replay setting, Online Curvature-Aware Replay [24]. Both, however, precondition the *entire* update, so the same metric that brakes the harmful push also brakes the restoring replay gradient, protecting the past task only by slowing the return to it. Asymmetric PER keeps the gate one-sided: it filters the new-task gradient alone and leaves the restoring gradient at full strength. We include the symmetric preconditioner, which damps both, as a control (Appendix C.4).

As detailed in Table 3.1, gates vary by their *domain*, scalar or directional, and their *control*: a feedback gate reacts to a live damage signal, while a feedforward gate commits in advance to a schedule or a precomputed filter. This thesis explores the two pure corners of this design space; a fixed linear ramp, the curriculum’s feedforward variant, serves as an ablation:

Adaptive λ -curriculum (Scalar Feedback): A scheduling parameter $\lambda(t)$ sets the new-task weight from the ratio of replay to new-task gradient norms, a live damage signal that fires whichever direction harm arrives from. By scaling the loss itself, it deforms the objective into the homotopy of Section 3.1, so the optimiser’s natural descent follows the reference path.

Asymmetric PER (Directional Feedforward): The gate $G = \delta(F_{\text{rep}} + \delta I)^{-1}$ is a fixed filter set by the replay Fisher, with the damping δ its single knob: as $\delta \rightarrow 0$ the push concentrates in the directions the past task is flat in. Committed in advance, it is feedforward, and therefore blind to damage arriving from directions the sampled Fisher misreads.

Method	Gate G	Domain	Control
Vanilla ER	I	—	—
GEM / A-GEM	reweight on conflict	scalar	feedback
MEGA	loss ratio	scalar	feedback
Balancing	norm matching	scalar	feedback
Linear curric.	$\lambda(t) I$, clock	scalar	feedforward
Adaptive curric.	$\lambda(t) I$, norm ratio	scalar	feedback
Asym. PER	$\delta(F_{\text{rep}} + \delta I)^{-1}$	directional	feedforward

Table 3.1: Replay methods as gates on the new-task gradient, Equation (3.3). The two methods of this thesis occupy the pure corners: a scalar feedback gate and a directional feedforward gate.

The taxonomy yields the hypotheses that Section 4 formalises as research questions. The scalar feedback

gate holds back the whole push and so pays a plasticity cost a separate accelerator must repay. The directional feedforward gate should protect the past task at little plasticity cost, but only within what its sampled curvature metric sees. The corners also differ in computational cost: the scalar gate adds only a gradient-norm ratio per step, while the directional gate solves a linear system in the replay Fisher at every step, by conjugate gradients over Fisher-vector products (Appendix A.4). Momentum is the natural accelerator, and it should discriminate between the corners. On vanilla ER it should leave the depth unchanged, because the unopposed first step at the boundary carries no accumulated velocity, while its velocity average smooths the noisy replay gradient and damps the chronic driver. The curriculum attenuates *at source*: with $\lambda \approx 0$ early there is nothing to accumulate, so momentum’s averaging and acceleration compose with the gate and repay its plasticity debt. The preconditioner attenuates each *step* while the gradient keeps its size, so velocity accumulated across steps can rebuild the displacement the filter removed, and momentum should partly undo the protection. How a gate composes with momentum is therefore set by where it attenuates, at the source or at the step, and a head-to-head comparison of the corners should depend on the momentum regime rather than trade along a fixed stability–plasticity frontier.

4 Research Questions

The two-level account and tracking bound of Section 3 make falsifiable predictions about the gap and each gate. The experiments are organised around four questions. The first three are answered on rotated MNIST with a small MLP, where the gap can be probed step by step across many conditions and seeds; the fourth asks whether those answers survive a deeper backbone and a stronger task shift.

RQ1 — The account. Does the gap separate into a deterministic arc of the descent path and overshoot of the followed path, such that removing the overshoot drivers leaves the arc, placing the optimiser on the reference path leaves the overshoot, and doing both removes the gap while the permanent stability–plasticity cost stays visible as reduced plasticity?

RQ2 — The scalar feedback gate. Does the λ -curriculum reduce the gap by timing the new-task push, and how do its ingredients divide the work? Four sub-questions probe the design:

RQ2a (linear ramp). Does gap depth fall as the ramp length N grows, consistent with the $O(1/N)$ bound?

RQ2b (adaptive schedule). Does the gradient-norm feedback signal match the best fixed ramp while removing the knob N ?

RQ2c ($\lambda_{\min}$ floor). Does a small floor $\lambda_{\min} > 0$ recover early plasticity at a modest depth cost?

RQ2d (momentum). Does momentum compose with the curriculum, closing the stochastic residual and restoring final accuracy, while leaving the depth of vanilla ER unchanged?

RQ3 — The directional feedforward gate. Does asymmetric PER reduce the gap as its filter strengthens, at what plasticity cost, and where does a feedforward gate fail? The account predicts two failure axes: blind spots of the sampled curvature metric, and momentum, whose accumulated velocity rebuilds the push the filter shrinks one step at a time.

RQ4 — Generalisation. Do the findings carry to a deeper backbone (ResNet-18) and a stronger task shift (CIFAR-10 domain corruption): does each gate still cut the gap at matched final accuracy, and do the momentum signatures of RQ2d and RQ3 reproduce there as a cross of momentum settings?

5 Methods

This section describes the empirical testbeds (Section 5.1), the metrics that summarise each run (Section 5.1.2), and the experimental blocks (Section 5.2) that answer the research questions of Section 4.

5.1 Experimental Setup

5.1.1 Testbeds

Two testbeds of different scale carry the study. A small, fast one (rotated MNIST on a two-layer MLP) runs the dense mechanistic sweeps: the driver block, the curriculum sweep, the asymmetric-PER damping sweep, and the momentum cross. A larger one (domain-shift CIFAR-10 on a CIFAR-adapted ResNet-18) tests whether the findings survive a deeper backbone and a stronger task shift, the regime where the local condition behind the bound of Section 3.1 weakens. Both hold the label space and task structure fixed and shift only the input distribution, and both headline on two tasks, so there is exactly one transition and one instance of the stability gap.

On rot-MNIST, following Lange et al. [16], T_0 presents the digits at -90° and T_1 applies a further 90° rotation, each task using the standard 60,000 training and 10,000 test images. On CIFAR-10, T_0 is clean and T_1 applies Gaussian-noise corruption at severity 3 [10]; the ten classes are shared, so only the input distribution shifts. A three-task variant

([none, Gaussian, shot]) and two alternative corruptions appear in the generalisation block. The rot-MNIST MLP is cheap enough to sweep across many conditions and seeds and is where the local condition of Section 3.1 is most plausible; the ResNet-18 is the deeper, more realistic counterpart. rot-MNIST runs online (one epoch per task), standard for stability-gap analysis; the ResNet needs several passes to converge before the transition, so the CIFAR block uses ten epochs per task. Optimiser, learning rate, and batch size are common across benchmarks. Full backbone and protocol specifications are in Appendix A.2 (Tables A.1–A.3).

Throughout training every task’s test accuracy is evaluated every 10 steps; at each transition the cadence rises to every step, so the dip is recorded at full resolution (the whole online epoch on rot-MNIST, a 250-step window on CIFAR). A stability-gap tracker snapshots each past task’s pre-switch accuracy and records the resulting (step, accuracy) series. For the driver block only, a gradient tracker additionally records the fidelity of the replay-buffer gradient against the exact past-task gradient, at the same cadence; each such step costs a full pass over the past-task training set, which is why it is not run elsewhere.

5.1.2 Metrics

Each metric is reported as the seed mean $\pm$ sample standard deviation; complete definitions are in Appendix B. The gap definition of Section 3.1, the transient excess below the level a task settles at, gives two primary metrics. With a_j^{pre} the pre-switch baseline on past task j and $a_j(t)$ its accuracy at step t , **gap_depth** is the maximum instantaneous excursion $\max_j(a_j^{\text{pre}} - \min_t a_j(t))$, which Section 3.2 predicts to be overshoot-dominated. The second, **gap_area_end**, integrates the shortfall below the settled level: with reference $b_j = \min(a_j^{\text{pre}}, \bar{a}_j)$, where $\bar{a}_j$ is the accuracy the task eventually reaches, it sums $\max(0, b_j - a_j(t))$ over the new task and across past tasks. Anchoring to the recovered level implements the split of Section 3.1: a permanent step down to a lower plateau contributes to the standard forgetting metrics and near-zero to the area, while a true dip-and-recover registers in full. One consequence, revisited when reading the tables, is that two methods settling at different levels are measured against different lines, so a better-recovering method can carry the *larger* area; a positive area marks transient disruption, not net cost, and we read it with final accuracy. The arc of Section 3.2 and the slow part of the noisy recovery live here; sharp noise-driven drops register in depth instead. A third metric, **recovery_steps**, and the gradient-fidelity diagnostics are defined in Appendix B.

To place the gap in the standard frame we report the continual-evaluation metrics of Lange et al. [16] from the task-boundary accuracy matrix: average fi-

nal accuracy (ACC), which checks that a gate lowers the gap at preserved accuracy; worst-case retention (min-ACC, WC-ACC); and short-horizon plasticity (WP₁₀₀). FORG and the windowed WF_w/WP_w family are logged for completeness. Conditions are compared with the paired Wilcoxon signed-rank test on seed-paired differences, preferred over the t -test because five paired seeds cannot support a normality assumption on bounded metrics: the gap metrics floor at zero, where the strongest conditions cluster, and accuracy sits near its ceiling, so the seed differences are skewed rather than Gaussian. With five paired seeds the smallest two-sided p -value is 0.0625, reached when all five differences share a sign; we read $p = 0.0625$ as complete directional agreement and let effect sizes separate large results from marginal ones.

5.2 Experiments

Each block maps to one or more research questions and tests the corresponding predictions of Section 3. All conditions use five seeds and the protocol above unless noted.

5.2.1 Overshoot Drivers (RQ1)

The driver block answers the overshoot side of RQ1: each condition switches off one of the drivers of Section 3.2 by construction and observes what remains. The complementary path-side intervention is the curriculum, whose full-buffer diagnostic below completes the RQ1 cross. Table 5.1 lists the four conditions. Balancing (D3, D4) gives g_{new} and g_{replay} equal magnitude from the first step, switching off the boundary imbalance: an unbalanced step at θ_0^* (the converged T_0 optimum, θ_A^* of Section 3) is driven almost entirely by g_{new} , a balanced step carries the replay direction at equal weight. The full-data buffer (D2, D4) replaces vanilla ER’s noisy 256-sample replay gradient with the exact empirical past-task gradient, switching off the estimator variance (mechanics in Appendix A.3). Because the drivers interact (Section 3.2), we read the conditions as interventions, not an additive decomposition: depth differences measure how much of the drop each driver carries in that context, while the area and per-step shape carry the level-one signal: with variance off (D2, D4), any dip-and-recover bend that remains is deterministic, the arc’s fingerprint.

5.2.2 The λ -Curriculum Sweep (RQ2, RQ1)

The curriculum sweep tests the scalar feedback gate one ingredient at a time: the $O(1/N)$ prediction of Equation (3.2) (RQ2a), the adaptive schedule (RQ2b), and the $\lambda_{\min}$ floor (RQ2c); a full-buffer diagnostic (C8) then returns to RQ1, testing the claim that the curriculum removes the arc and leaves only stochastic overshoot. All conditions run on top of standard ER (1k reservoir, unbalanced gradients), so

the schedule is the only changed variable (Table 5.2). The adaptive schedule (C4) is the feedback variant: it sets λ to a clipped exponential moving average of the gradient-norm ratio $r := \|g_{\text{replay}}\|/\|g_{\text{new}}\|$. At θ_0^* the replay gradient is near zero by stationarity, so r and λ start near 0; as the iterate moves, $\|g_{\text{replay}}\|$ grows and $\|g_{\text{new}}\|$ shrinks, so $\lambda \rightarrow 1$ at the pace the damage signal dictates, with no ramp length to choose. The floor $\lambda_{\min}$ (C5–C7) keeps the schedule off $\lambda \approx 0$ during the first steps and buys back early plasticity.

C8 combines the shipping curriculum (C7) with the full 60k buffer of D2/D4, so the path is fixed and the variance driver is switched off at source. This makes it a double probe. First, it tests what C7’s remaining momentum-0 transient is made of: the curriculum has already removed the arc, so if the residual is stochastic overshoot from buffer noise, the exact replay gradient should remove it too and C8 should show no gap even without momentum. Second, comparing C8 with and without momentum (C8 vs. C8_M) separates the two roles momentum could play for the curriculum. The full buffer already removes the gradient noise, so C8 has no variance left for momentum to reduce; whatever momentum still improves on top of C8 must come from its other role, accelerating new-task learning. C8 stores the whole past task and so breaks the limited-memory premise; it is a diagnostic only, and C7 remains the practical configuration.

5.2.3 Momentum Cross (RQ2d, RQ3)

Every rot-MNIST condition (the driver D-series, the curriculum C-series, and the asymmetric-PER sweep legs P1–P2 of Table 5.3) is run at momentum 0.0 and 0.9; momentum-on runs carry an M subscript (e.g. D1_M). The cross tests the composability prediction of Section 3.3: whether momentum helps or hurts a method’s gap should depend on where the method attenuates the push.

5.2.4 Asymmetric-PER δ Sweep (RQ3)

The damping sweep tests the directional feedforward gate: does the gap shrink as the filter strengthens, at what plasticity cost, and do the two predicted feedforward failures appear? Asymmetric PER implements Equation (1.2), filtering the new-task push through the replay Fisher F_{rep} [12]. Normalising by δ keeps flat directions at unit gain: along an eigendirection of curvature ρ the gain is $\delta/(\rho + \delta)$, so directions the past task is flat in pass through whole and steep ones are shrunk toward zero. The damping δ is the single knob: at $\delta \rightarrow \infty$ plain ER returns, while as $\delta \rightarrow 0$ the push concentrates in replay-flat directions with the restoring gradient g_{rep} at full strength throughout. The push is applied matrix-free by conjugate gradients over Fisher-vector products (Appendix A.4).

Table 5.3 lays out the sweep: $\delta \in \{1.0, 0.3, 0.1, 0.03\}$ crossed with two buffer legs, each at both momen-

Label	Condition	Replay buffer	Gradient handling		Driver removed by construction
D1	Vanilla ER	1k reservoir	unbalanced		none (all drivers active)
D2	Full-data ER	60k (all past)	unbalanced		estimator variance (exact replay gradient)
D3	Balanced ER	1k reservoir	unit-norm	balanced	boundary imbalance
D4	Full-data + balanced	60k	unit-norm	balanced	imbalance and variance

Table 5.1: Driver-removal conditions. Each switches off one or two overshoot drivers relative to vanilla ER; what all four leave in place is the descent path itself, so the arc should survive throughout.

Label	Condition	$\lambda(t)$ schedule	Hyperparameter	Tests
—	Vanilla ER (baseline)	$\lambda \equiv 1$ from step 1 (abrupt switch)	—	reference
C1	Linear $N=50$	$\text{clip}(t/50, 0, 1)$	$N = 50$	RQ2a
C2	Linear $N=100$	$\text{clip}(t/100, 0, 1)$	$N = 100$	RQ2a
C3	Linear $N=200$	$\text{clip}(t/200, 0, 1)$	$N = 200$	RQ2a
C4	Adaptive	$\text{clip}(\text{EMA}_\alpha(r), 0, 1)$	$\alpha = 0.05$	RQ2b
C5	Adaptive + $\lambda_{\min}=0.05$	$\max(\lambda_{\min}, \text{clip}(\text{EMA}_\alpha(r), 0, 1))$	$\alpha = 0.05, \lambda_{\min}=0.05$	RQ2c
C6	Adaptive + $\lambda_{\min}=0.10$	as C5	$\lambda_{\min}=0.10$	RQ2c
C7	Adaptive + $\lambda_{\min}=0.20$	as C5	$\lambda_{\min}=0.20$	RQ2c
C8	C7 + full 60k buffer	as C7, exact replay gradient	diagnostic	RQ1, RQ2d

Table 5.2: λ -curriculum conditions, all applied on top of standard ER. EMA_α is an exponential moving average with smoothing α , and $r := \|g_{\text{replay}}\|/\|g_{\text{new}}\|$ is the gradient-norm ratio. Each condition is run at momentum 0.0 and 0.9.

Label	Condition	Replay buffer	Damping δ	Purpose
P1	Standard leg	1k reservoir	1.0, 0.3, 0.1, 0.03	practical regime, matched to the curriculum (RQ3)
P2	Full-buffer leg	60k (exact g_{rep})	1.0, 0.3, 0.1, 0.03	variance driver off; residual area is the arc (RQ3, RQ1)
P3	Solver control	60k	0.03	60 CG iterations instead of 25; rules out solve artefacts
P4	Symmetric control	1k reservoir	1.0, 0.3, 0.1, 0.03	preconditions $g_{\text{cur}} + g_{\text{rep}}$; ablates the one-sided gate

Table 5.3: Asymmetric-PER conditions. Both sweep legs run at momentum 0.0 and 0.9; the two controls run at momentum 0, where the effects they check for arise. The filter metric is always the replay Fisher F_{rep} , estimated on a sampled batch (Appendix A.4).

tum settings. The *standard* leg (1k reservoir) matches the curriculum’s practical regime, while the *full-buffer* leg makes the replay gradient exact, switching off the variance driver as in D2/D4, so the remaining area measures the deterministic arc and the sweep reads out directly whether the gate shrinks the arc itself. Two controls guard the reading: a solver control re-runs the strongest filter at 60 CG iterations to separate gate properties from an under-converged solve (Appendix A.4), and a symmetric control preconditions the summed gradient by the same replay Fisher, isolating the effect of leaving the restoring gradient untouched (Appendix C.4).

5.2.5 Generalisation: CIFAR-10 (RQ4)

The CIFAR-10 block tests whether both gates and the momentum signatures carry to a deeper backbone and a stronger task shift. All CIFAR conditions use the

ResNet-18 protocol of Table A.3 (ten epochs per task, buffer 5,000). The headline conditions run at momentum 0.9 (Table 5.4, top), each method crossed with BatchNorm and GroupNorm because the two respond differently at the switch: the input shift disrupts BatchNorm’s running statistics, which must re-adapt during the first steps, while GroupNorm uses no cross-batch statistics, so its transient reflects optimisation dynamics alone. The curriculum ships as the adaptive schedule at $\lambda_{\min} = 0.20$ and asymmetric PER at $\delta = 0.1$, a strong rot-MNIST setting clear of the blind-spot regime; both carry their rot-MNIST configurations without a CIFAR-specific sweep, so they meet the deeper backbone on equal footing. A momentum cross (Table 5.4, bottom) adds momentum-0 runs of all three methods at GroupNorm, repeating the three-way composability test where the transient is free of normalisation-statistics effects; because momentum also drives convergence on this backbone, accuracy is

compared within a momentum setting. A further generalisation block pairs the shipping curriculum against a paired vanilla-ER control on shifts it was never tuned on (a two-task shot-noise shift, a two-task contrast shift, and a three-task [none, Gaussian, shot] sequence, each at both normalisations over three seeds) to test whether the result depends on the particular corruption or on having a single transition.

Label	Method	Norm	μ
G1	Vanilla ER	BatchNorm	0.9
G3	Adaptive $\lambda_{\min}=0.20$	BatchNorm	0.9
G4	Vanilla ER	GroupNorm	0.9
G6	Adaptive $\lambda_{\min}=0.20$	GroupNorm	0.9
G7	Asym. PER $\delta=0.1$	BatchNorm	0.9
G8	Asym. PER $\delta=0.1$	GroupNorm	0.9
G9	Vanilla ER	GroupNorm	0.0
G10	Adaptive $\lambda_{\min}=0.20$	GroupNorm	0.0
G11	Asym. PER $\delta=0.1$	GroupNorm	0.0

Table 5.4: CIFAR-10 conditions (two-task Gaussian-noise shift, buffer 5,000, five seeds). Top: the headline block at momentum 0.9. Bottom: the momentum cross, pairing momentum-0 runs of all three methods against their momentum-0.9 counterparts at GroupNorm. Labels G2 and G5 are the NCL baseline at each normalisation, reported in Appendix C.5. The generalisation block (novel corruptions and the three-task sequence, three seeds) is described in the text and reported in Appendix C.6.

6 Results

The results follow the research questions: the two-level account (Section 6.1, RQ1), the scalar feedback gate (Section 6.2, RQ2), the directional feedforward gate (Section 6.3, RQ3), a synthesis of the two momentum crosses (Section 6.4), and the CIFAR-10 generalisation of both gates (Section 6.5, RQ4).

Reading the tables. Depth is in percentage points (pp): a depth of 19.5 means the worst past-task evaluation falls 19.5 points below its pre-switch baseline. Section 3.2 predicts depth to be overshoot-dominated, and both overshoot drivers express in it: the acute boundary spike, and the buffer’s noise, whose sudden drops can set the worst evaluation long after the switch. The end-referenced area `gap_area_end` (accuracy-steps) integrates the excursion below the level each task settles at, so a permanent drop scores near zero and contributes instead to forgetting. Because each condition’s area is taken against its own settled level (Section 5.1.2), area is comparable only between conditions that recover to the same level, so we read cross-condition area together with final accuracy and the per-step curves. Values are mean $\pm$ standard deviation over five seeds (three for the CIFAR generalisation block). Following Section 5.1.2,

the two-sided Wilcoxon test floors at $p = 0.0625$ when all five seed differences share a sign, so we quote a p -value only where the seeds disagree and otherwise report complete directional agreement. The full metric set for every condition is in Appendix C.

6.1 The Two Levels, Separated (RQ1)

RQ1 asks whether the gap separates into a deterministic arc of the descent path and overshoot of whatever path is followed, such that removing the overshoot drivers and fixing the path are separately insufficient but jointly sufficient. Crossing the two interventions at momentum 0 gives the 2×2 of Table 6.1: the driver-removal conditions D1–D4 fix nothing about the path, and the curriculum conditions C7/C8 fix the path while leaving or removing the variance driver.

	Drivers present	Drivers removed
Bowed path (no gate)	D1: 19.5 / 6.3 / 0.873	D4: 1.9 / 3.2 / 0.869
Valley-floor (curriculum)	C7: 13.9 / 2.7 / 0.837	C8: 0.2 / 0.05 / 0.818

Table 6.1: The RQ1 2×2 on rot-MNIST at momentum 0, each cell reporting `gap_depth` (pp) / `gap_area_end` / ACC. Rows cross the path (bowed steepest descent vs. the valley-floor path the curriculum follows); columns cross the overshoot drivers (present vs. removed by balancing and the exact full-buffer replay gradient). Removing the drivers alone (D4) leaves a deterministic arc; fixing the path alone (C7) leaves the overshoot; doing both (C8) closes the gap, while the permanent stability-plasticity cost stays visible as the lowest ACC in the table.

Removing the drivers leaves the arc. Following the driver ladder (D1–D4, full metrics in Appendix C.1, Table C.1), unit-norm balancing (D3) removes the boundary imbalance and takes 13.8 pp off the depth ($19.5 \rightarrow 5.7$), by far the largest single drop; swapping the reservoir for the exact past-task gradient (D4) removes the estimator variance and takes off a further 3.8 pp ($5.7 \rightarrow 1.9$). Gradient-fidelity diagnostics confirm the driver each condition switches off: the finite-buffer D1 and D3 reach only $\text{c\ddot{o}s}(g_{\text{replay}}, g_{\text{true}}) \approx 0.71$ and 0.66 with per-step minima near zero, while the full-data D2 and D4 sit at 1.000 by construction (Appendix C.3). What no driver removal touches is the descent path, and the area shows it: with the spike gone at D4 (only 1.9 pp) a clear bend remains, `gap_area_end` = 3.2 (the D4 cell of Table 6.1). Because the full-data conditions carry zero gradient noise, that area cannot be stochastic residual; it is the deterministic arc of Section 3.2. Figure 6.1 shows it directly: the exact-gradient curves (D2, D4) trace a smooth dip-and-recover with no jitter, the fingerprint the account attaches to a bowed path rather than a spike.

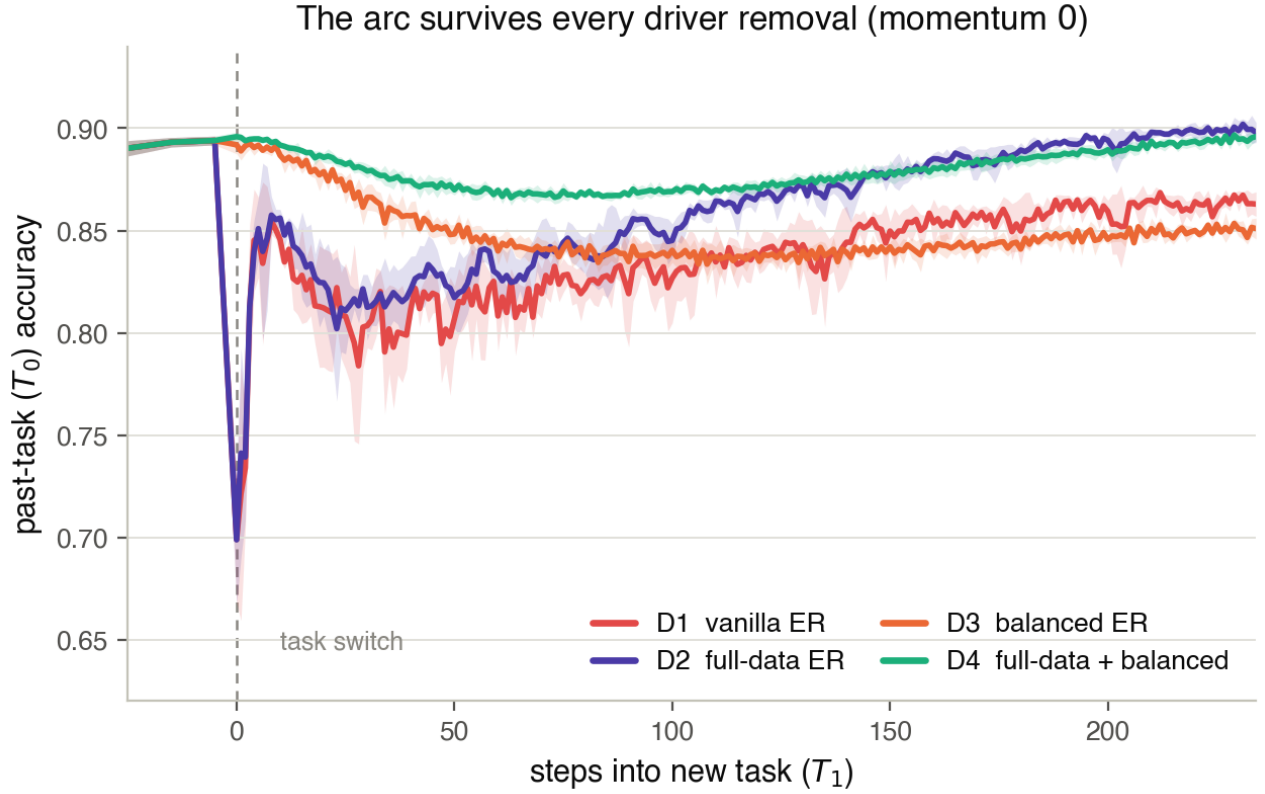


Figure 6.1: Per-step past-task (T_0) accuracy across the $T_0 \rightarrow T_1$ transition for the four driver-removal conditions at momentum 0; solid lines are the seed mean, bands the standard deviation, $x=0$ the task switch. *Depth* of the boundary spike falls when balancing switches off the imbalance driver (deep for the unbalanced D1, D2; shallow for the balanced D3, D4), while *jitter* vanishes when the exact replay gradient switches off the variance driver (the full-data D2, D4 are smooth, the finite-buffer D1, D3 noisy). What no removal touches is the smooth dip-and-recover *bend* beneath the spike, cleanest in the exact-gradient D4 where no jitter can hide it: that bend is the deterministic arc, and it responds to no optimiser hygiene, only to a change of path.

Fixing the path leaves the overshoot. The other row holds the drivers in place and fixes the path with the curriculum. At momentum 0 the shipping curriculum (C7) still shows 13.9 pp of depth (Table 6.1): with the arc removed, essentially all of it is overshoot. That this residual is estimator variance rather than anything structural is settled by C8, which adds the exact full buffer to C7 so that the path is fixed *and* the variance driver is switched off at source. The gap then collapses to 0.2 ± 0.3 pp of depth and 0.05 of area, the smallest of any condition in the study, and the C7→C8 area drop ($2.7 \rightarrow 0.05$) holds across all five seeds. Overshoot is real and untouched by the path fix; the arc is real and untouched by the driver removal; only doing both closes the gap.

The permanent cost stays on the ledger. Closing the transient does not make the transition free. C8’s final accuracy is 0.818, the lowest in Table 6.1 and 0.055 below vanilla ER, because holding the new-task push down at momentum 0 slows the new task without an accelerator to repay it. This is the permanent stability–plasticity cost the account predicts to remain once the transient is removed; Section 6.2 shows momentum paying it back. A second, independent line of evidence that the arc is deterministic geometry comes from the directional gate (Section 6.3): on its full-buffer legs, where the variance driver is off and the area *is* the arc, the area shrinks monotonically as the filter strengthens ($2.3 \rightarrow 1.0 \rightarrow 0.4$ for $\delta = 1.0 \rightarrow 0.3 \rightarrow 0.1$). The arc responds to path-level intervention and to nothing else, whether the intervention times the push (the curriculum) or filters its direction (the gate).

6.2 The Scalar Feedback Gate (RQ2)

Table 6.2 reports the curriculum sweep at both momentum settings against vanilla ER. The momentum-off leg tests the schedule in isolation; the momentum-on leg is the shipping regime.

Lengthening the ramp strictly reduces gap depth (RQ2a–c). At momentum 0, gap depth falls monotonically as the linear ramp lengthens, as the $O(1/N)$ bound of Equation (3.2) predicts (RQ2a), though successive lengths lie within the seed spread and the reduction is bought at a real accuracy cost: by $N = 200$ ACC has fallen from 0.873 to 0.828 and the tail has stretched, `recovery_steps` growing from 36 to 133 (Appendix C). The adaptive schedule (C4) reaches 12.8 pp, below the best linear ramp, without a ramp length to set (RQ2b; C3→C4 agrees in direction but stays within noise, $p = 0.44$). Raising the floor $\lambda_{\min}$ to 0.20 (C7) shifts the schedule in the predicted Pareto direction only weakly at momentum 0, because early T_1 progress is then set by the large

deterministic boundary gradient the small floor cannot redirect (RQ2c). The momentum-0 curves of Figure 6.2 (dashed) show the trade directly: the curriculum reaches a shallower past-task dip than vanilla (a) but pays for it in slower early T_1 learning (b).

Momentum pays the plasticity debt, and closes the gap (RQ2d). The momentum-on leg is where the scalar gate ships, and it acts through momentum’s two roles at once. Adding momentum to the adaptive schedule drops depth to 2.5 pp (C4→C4_M) and to 3.7 pp at the $\lambda_{\min} = 0.20$ floor (C7_M), collapses the area to near zero, and restores accuracy to vanilla ER’s level (Table 6.2); every one of these contrasts holds across all five seeds. The depth and area collapse is variance reduction: momentum averages the noisy finite-buffer replay gradient across accumulated velocity, damping the chronic driver that feeds the residual dip and tail once the schedule has tempered the acute boundary push at source. The accuracy restoration is the second, separate role, acceleration, driving the new task hard enough to repay the plasticity the schedule borrows. The floor that was inert at momentum 0 becomes a clean Pareto knob once momentum is active: raising it from 0 to 0.20 (C4_M → C7_M) trades 1.2 pp of depth for 0.014 of accuracy and a gain in short-horizon plasticity (WP₁₀₀; Appendix C), because a higher floor lets T_1 train from the first post-switch step. At C7_M the past task no longer dips and recovers but settles at a marginally lower plateau (area ≈ 0), so the residual 3.7 pp is the permanent accommodation, not a transient. This configuration ships to CIFAR.

That momentum here closes a gap it left untouched under vanilla ER is the composability signature of Section 5.2.3, and a full-buffer diagnostic separates its two roles cleanly (Table C.3, Appendix C.2): with the full buffer standing in for momentum’s averaging, C8 already has essentially no gap at momentum 0 (0.2 pp), so the residual C7 leaves is variance, closable by source variance reduction rather than by acceleration; adding momentum to C8 then lifts ACC to 0.955 (the highest in the benchmark) while re-introducing only a 2.0 pp transient. The headline is that *the curriculum needs momentum for accuracy, not for the gap*. The full 60k buffer defeats the limited-memory premise and is diagnostic only; the practical configuration is C7_M (3.7 pp / 0.035 / 0.931) at 1/60th the memory. For completeness the cross also confirms the vanilla-ER null: momentum shifts vanilla depth only from 19.5 to 15.9 pp, smaller than the seed spread with seeds split on its sign ($p = 0.44$), because the deterministic unopposed first step carries no accumulated velocity to average; it does cut vanilla’s tail (area $6.3 \rightarrow 1.0$, all seeds). Momentum leaves vanilla’s *depth* intact while smoothing its recovery, not the gap.

Condition	Momentum 0.0			Momentum 0.9		
	gap_depth (pp)	gap_area_end	ACC	gap_depth (pp)	gap_area_end	ACC
Vanilla ER (baseline)	19.5 ± 4.1	6.3 ± 0.5	0.873 ± 0.005	15.9 ± 2.6	1.0 ± 0.2	0.928 ± 0.003
C1: Linear $N=50$	17.5 ± 1.7	4.9 ± 0.5	0.851 ± 0.023	6.4 ± 0.5	0.2 ± 0.1	0.932 ± 0.003
C2: Linear $N=100$	16.2 ± 4.3	3.6 ± 0.3	0.844 ± 0.025	6.4 ± 0.6	0.2 ± 0.1	0.932 ± 0.003
C3: Linear $N=200$	14.3 ± 4.6	1.8 ± 0.6	0.828 ± 0.025	6.2 ± 0.6	0.1 ± 0.0	0.925 ± 0.007
C4: Adaptive	12.8 ± 2.9	2.2 ± 0.5	0.833 ± 0.025	2.5 ± 0.5	0.3 ± 0.1	0.917 ± 0.001
C5: Adaptive $\lambda_{\min}=0.05$	13.2 ± 3.2	2.2 ± 0.5	0.835 ± 0.025	2.6 ± 0.5	0.1 ± 0.0	0.922 ± 0.001
C6: Adaptive $\lambda_{\min}=0.10$	13.9 ± 3.3	2.4 ± 0.5	0.836 ± 0.026	3.0 ± 0.4	0.0 ± 0.0	0.927 ± 0.002
C7: Adaptive $\lambda_{\min}=0.20$	13.9 ± 3.4	2.7 ± 0.6	0.837 ± 0.031	3.7 ± 0.4	0.0 ± 0.0	0.931 ± 0.002

Table 6.2: λ -curriculum sweep on rot-MNIST against vanilla ER, at momentum 0.0 and 0.9; five seeds. The scalar feedback gate reaches a few points of depth at matched accuracy only in the momentum-on regime it ships in.

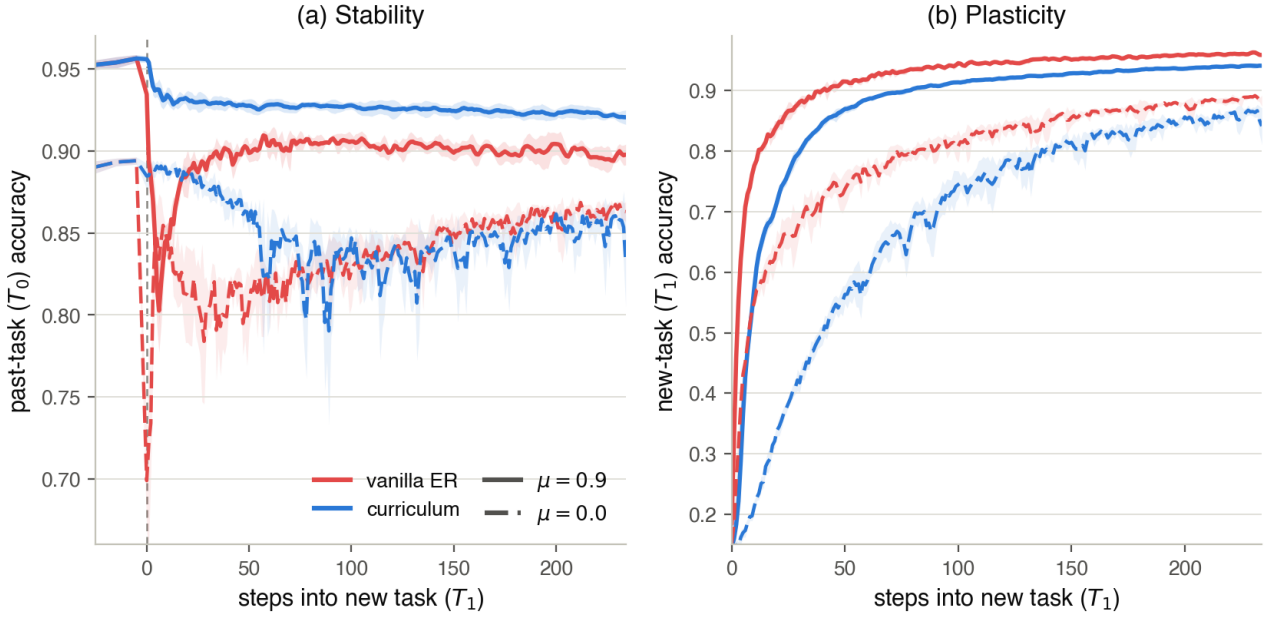


Figure 6.2: The scalar feedback gate and momentum’s two roles: per-step accuracy across the rot-MNIST switch, seed mean with $\pm$ std bands, for vanilla ER (red) and the shipping adaptive curriculum $\lambda_{\min}=0.20$ (blue), each at momentum 0.9 (solid) and 0.0 (dashed). (a) Stability (T_0): only curriculum-with-momentum (solid blue) holds the past task near its plateau through the switch; momentum barely dents vanilla’s spike (solid red still dives), and both methods open a deep, jittery gap without it (dashed). Momentum closes the gap where the gate has already tempered the push at source, not otherwise. (b) Plasticity (T_1): at momentum 0 the curriculum slows early new-task learning (dashed blue is slowest to rise — the plasticity debt of holding the push down), but momentum pays it back, lifting the solid-blue curve back toward the vanilla pace.

6.3 The Directional Feedforward Gate (RQ3)

RQ3 asks whether asymmetric PER shrinks the gap as its filter strengthens ($\delta \rightarrow 0$), at what plasticity cost, and where a feedforward gate fails. Table 6.3 reports the δ sweep on both buffer legs at momentum 0.

Monotone, and free. On the standard buffer, depth falls from 9.0 to 4.8 pp as δ goes from 1.0 to 0.03 while ACC *rises* from 0.869 to 0.880 and WP_{100} from 0.733 to 0.741; every one of these trends holds across all five seeds (Figure 6.3(a)). The roughly 4 pp of accuracy the scalar gate pays at momentum 0 never materialises here, because the filter blocks the push only in the directions the past task is steep in and lets it through everywhere else. This is the scalar gate’s weakness inverted, and it is what makes the two gates complements rather than a strong and a weak version of the same thing. The full-buffer leg isolates the arc: with the variance driver off, the area shrinks with the filter, $2.3 \rightarrow 1.0 \rightarrow 0.4$ for $\delta = 1.0 \rightarrow 0.3 \rightarrow 0.1$ (the $1.0 \rightarrow 0.3$ drop holds across all seeds), the independent level-1 evidence promised in Section 6.1: the directional gate bends the followed path back toward the reference, as the curriculum does by another route. The strongest clean setting, $\delta = 0.3$ full buffer, reaches 1.5 pp of depth, undercutting the driver floor D4 at the best momentum-0 accuracy in the benchmark (0.900). Leaving the restoring gradient g_{rep} at full strength is what makes this a net removal of damage rather than a reshuffling of it: a symmetric variant that preconditions the summed update strangles the restoring gradient harder than the harmful push, so its forgetting *climbs* as the filter strengthens (Appendix C.4, Table C.5).

At momentum 0 the directional gate dominates the scalar gate. Head to head at momentum 0, the strongest standard-buffer setting ($\delta=0.03$: 4.8 pp / 0.880) beats the shipping curriculum (C7: 13.9 pp / 0.837) on both axes at once, lower in depth and higher in accuracy. Part of this is that the curvature filter also damps how far a noisy gradient moves the past-task loss, suppressing the per-step *expression* of variance where the scalar gate needs momentum or a larger buffer. The scalar gate’s case is not this corner; it is momentum composability and blind-spot robustness, which the next two results show the directional gate lacking.

The cliffs: a feedforward blind spot. The one place the sweep breaks monotonicity is the strongest full-buffer setting, $\delta = 0.03$, where mean depth jumps to 9.5 pp with a std of 8.2: in 3 of 5 seeds the past-task curve, having held flat for two hundred steps, slides off a late cliff (*recovery_steps* ≈ 217 ; min-ACC std

0.094), the red per-seed traces in Figure 6.3(b). Re-running the same setting with the conjugate-gradient solver taken from its 25-iteration baseline to 60 reproduces the cliffs at the same seeds and steps and at a comparable mean depth (11.5 pp, a separate confirmation value alongside the 9.5 pp of the baseline in Table 6.3), so the cliffs are not an artefact of an under-converged solve. They are the feedforward failure the taxonomy predicts: new-task pressure escapes through directions the *sampled* Fisher rates as flat but the true past-task loss punishes further out.* A gate that brakes only where its model of the past task says to is blind wherever that model is wrong, and the model is estimated on finite data under a saturating loss. The cliffs are the empirical case for a feedback signal, which reads the damage whatever direction it arrives from.

Momentum re-inflates the per-step filter. Momentum undoes the directional gate’s protection, the opposite of what it does for the scalar gate. On the full buffer, momentum lifts depth from 2.2 to 7.6 pp at $\delta=0.1$ and from 1.5 to 9.1 pp at $\delta=0.3$ (both across all five seeds), while accuracy jumps to ≈ 0.965 . Momentum accumulates the already-filtered direction, but the filter attenuates the step, not the source: the new-task loss, and with it the per-step gradient, keeps its full size, and because motion in the braked directions is slow, the shrunk push persists there step after step. A persistent push entering the accumulator is amplified toward $1/(1 - \mu)$ times its per-step size, so velocity rebuilds displacement in the very directions the filter shrinks one step at a time. Under momentum, asymmetric PER becomes the highest-accuracy replay method in the study but at a depth (≈ 8 pp) between the scalar gate’s (3.7 pp, C7_M) and vanilla ER’s (15.9 pp). This re-inflation, not a fixed ordering, is what makes the head-to-head comparison of the next subsection regime-dependent.

6.4 Reading the Momentum Crosses Together

This subsection answers no separate research question; it reads the momentum crosses of RQ2d and RQ3 side by side. Together they show that the two gates do not sit on a fixed stability–plasticity frontier that a practitioner trades along: which gate leaves the smaller gap is decided by the momentum setting, the same setting that decides final accuracy. An earlier reading of these results as a frontier was an artefact of comparing the gates at mismatched buffer

*A finer reading—that softmax saturation on replay data drives the local Fisher toward zero while a cliff lies ahead, and that the 1k buffer’s sampling noise incidentally dithers such escapes away, which is why the cliffs appear on the full-buffer leg—is consistent with the data but not established here; we report the failure at the blind-spot level the solver control supports.

δ	Standard (1k) buffer				Full (60k) buffer			
	depth (pp)	area	ACC	WP ₁₀₀	depth (pp)	area	ACC	min-ACC
1.0	9.0 ± 0.6	3.3	0.869	0.733	4.0 ± 1.0	2.3	0.898	0.841
0.3	7.4 ± 1.1	2.3	0.873	0.736	1.5 ± 1.1	1.0	0.900	0.867
0.1	5.9 ± 1.5	1.6	0.876	0.739	2.2 ± 1.6	0.4	0.901	0.860
0.03	4.8 ± 1.9	1.0	0.880	0.741	9.5 ± 8.2	0.4	0.903	0.787

Table 6.3: Asymmetric-PER δ sweep on rot-MNIST at momentum 0, five seeds. Depth and area fall as $\delta \rightarrow 0$ while ACC and WP₁₀₀ *rise*: the directional gate protects the past task at no plasticity cost. On the full buffer, where the area is the deterministic arc, the arc shrinks with the filter, and the strongest clean setting ($\delta=0.3$) undercuts the driver-removal floor (D4: 1.9 pp). The far cell is the exception: at $\delta=0.03$ full buffer the mean depth jumps to 9.5 pp with a large std, the cliffs of the text.

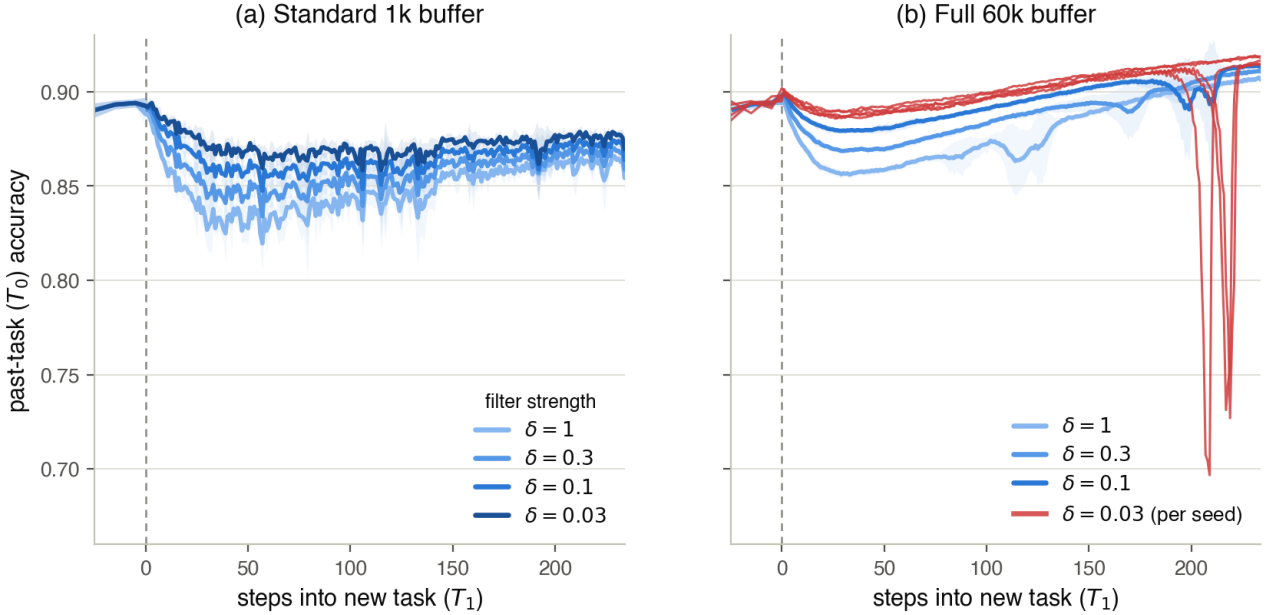


Figure 6.3: The directional feedforward gate: per-step past-task (T_0) accuracy across the rot-MNIST switch at momentum 0, sweeping the damping δ (weaker filter light $\rightarrow$ stronger filter dark). (a) Standard 1k buffer: the boundary dip shrinks monotonically as $\delta \rightarrow 0$ — and accuracy rises with it (Table 6.3) — so the protection is bought at no plasticity cost. (b) Full 60k buffer (variance driver off, so the residual excursion is the deterministic arc): the arc shrinks with the filter for $\delta \geq 0.1$, but at the strongest setting $\delta=0.03$ the feedforward gate hits its curvature blind spot — shown per seed (critical red), 3 of 5 seeds hold flat for ~ 200 steps and then slide off a late cliff, the failure a feedback signal cannot be blind-sided into.

sizes: the directional gate’s headline momentum accuracies (≈ 0.965) come from its full 60k-buffer legs, set against the curriculum’s practical 1k buffer, so the “trade” was really a memory-budget difference in disguise. Held at a common buffer, the frontier collapses to a regime dependence.

No momentum: the directional gate is strictly better. At momentum 0 and a matched 1k buffer, the directional gate dominates the scalar gate on both axes. Its strongest clean setting reaches 4.8 pp of depth at 0.880 accuracy (Table 6.3) against the curriculum’s 13.9 pp at 0.837 (C7, Table 6.2): lower gap *and* higher accuracy, because the per-step filter protects the past task without ever slowing the new one at source, so it carries no plasticity debt for the momentum-0 regime to leave unpaid. Where retention is the only concern and momentum is off, there is no reason to prefer the scalar gate.

Momentum: the scalar gate takes over. Momentum is required for accuracy on any realistic backbone, and it reverses the ranking through the mechanism of Section 6.3: it re-inflates the directional gate’s per-step filter, rebuilding the push the filter removes one step at a time, so the directional gate’s depth climbs back to ≈ 8 pp. The scalar gate does the opposite, composing with momentum because it attenuated at source, and reaches 3.7 pp at vanilla accuracy (C7_M). Once the buffers match, the directional gate has no accuracy advantage left to offset its re-inflated gap; the scalar gate is simply the lower-gap method in the regime that ships. The choice between the gates is therefore not a point on a frontier but a consequence of whether momentum is on: off, take the directional gate; on, take the scalar gate.

6.5 Generalisation to CIFAR-10 (RQ4)

The CIFAR-10 block tests whether both gates carry to a ResNet-18, a clean $\rightarrow$ corruption shift, and the two normalisations that govern the post-switch recovery. Table 6.4 reports the headline block at momentum 0.9, the shipping regime.

Both gates carry to the deeper backbone. The rot-MNIST result carries over and strengthens. Under batch norm the curriculum cuts depth from 11.8 to 4.8 pp and asymmetric PER to 5.9 pp; under group norm, where vanilla ER suffers a catastrophic 41.2 pp drop, the curriculum cuts it to 5.2 pp and asymmetric PER to 5.6 pp. Every gate-vs-vanilla depth and area contrast holds across all five seeds, at final accuracy within about 0.01 of vanilla and with worst-case retention sharply improved, min-ACC rising from 0.31 to 0.67 under group norm. On this backbone the transient stays open for hundreds of steps, so the area

carries as much of the comparison as the depth (Figure 6.4): vanilla ER under group norm crashes T_0 almost to chance and crawls back over the whole recovery window, while both gates hold T_0 near its plateau through the switch. At the matched 5,000 buffer and momentum 0.9 both gates land in the same few-pp band, with the curriculum the cheaper of the two (below), consistent with the momentum regime of Section 6.4.

Mechanism, generalisation, and cost. Three further checks are reported in full in Appendix C.7. A momentum cross on the deep backbone (Table C.8) reproduces two of the three predicted signatures — momentum leaves vanilla depth unchanged ($40.5 \rightarrow 41.2$ pp) and composes with the curriculum ($28.6 \rightarrow 5.2$ pp) — while the directional-gate re-inflation is confounded by momentum-0 under-convergence of the ResNet (vanilla min-ACC 0.11), so the re-inflation signature rests on the rot-MNIST result where the full-buffer MLP isolates it cleanly. A generalisation block (Appendix C.6, Table C.7) shows the curriculum beating a paired vanilla control on two corruptions it was never tuned on (shot noise $46.3 \rightarrow 7.3$ pp, contrast $8.4 \rightarrow 3.9$ pp under group norm), with a single three-task group-norm sequence flagged as a reliability boundary of the adaptive ratio when two consecutive corruptions are near-identical and both gradient norms go small. Finally, compute separates the gates where retention does not (Table C.9): the scalar gate is one gradient-norm ratio per step and its wall-clock is indistinguishable from vanilla ER, while the directional gate’s 25 conjugate-gradient Fisher-vector products per step run up to $4\times$ slower on the ResNet-18, a multiple that grows with backbone depth. In the shipping momentum regime, then, the scalar gate matches the directional gate’s retention at vanilla-ER cost and without the blind-spot cliffs, which is why it is the default recommendation.

6.6 Summary of Findings

The ladder holds end to end. (i) The gap separates into a deterministic arc and overshoot of the followed path: removing the drivers leaves the arc (D4: 1.9 pp, area 3.2), fixing the path leaves the overshoot (C7: 13.9 pp), doing both closes the gap (C8: 0.2 pp), while the permanent stability-plasticity cost stays on the ledger (C8 ACC 0.818). (ii) The scalar feedback gate times the push down the valley-floor path and, with momentum, reaches 3.7 pp of depth at vanilla accuracy; momentum closes its gap by averaging variance and pays back the plasticity the schedule borrows, so the curriculum needs momentum for accuracy, not for the gap. (iii) The directional feed-forward gate shrinks depth and arc monotonically as $\delta \rightarrow 0$ at no plasticity cost, but has curvature blind spots (the $\delta=0.03$ cliffs, solver-controlled) and

Norm	Method	gap_depth (pp)	gap_area_end	ACC	min-ACC
BatchNorm	G1: Vanilla ER	11.8 ± 2.5	17.5 ± 4.6	0.723 ± 0.008	0.644 ± 0.023
	G3: Curriculum ($\lambda_{\min}=0.20$)	4.8 ± 0.7	3.6 ± 2.2	0.722 ± 0.011	0.713 ± 0.017
	G7: Asym. PER ($\delta=0.1$)	5.9 ± 4.3	3.3 ± 3.0	0.715 ± 0.011	0.698 ± 0.023
GroupNorm	G4: Vanilla ER	41.2 ± 13.5	34.9 ± 23.7	0.730 ± 0.011	0.309 ± 0.111
	G6: Curriculum ($\lambda_{\min}=0.20$)	5.2 ± 2.1	1.7 ± 0.9	0.731 ± 0.020	0.670 ± 0.055
	G8: Asym. PER ($\delta=0.1$)	5.6 ± 1.1	4.8 ± 3.8	0.741 ± 0.009	0.673 ± 0.023

Table 6.4: CIFAR-10 headline block (ResNet-18, momentum 0.9, buffer 5,000, five seeds). Both gates cut gap depth and area far below vanilla ER at matched final accuracy and much higher worst-case retention (min-ACC), under both normalisations. Neither was given a CIFAR-specific sweep; both carry their rot-MNIST configurations.

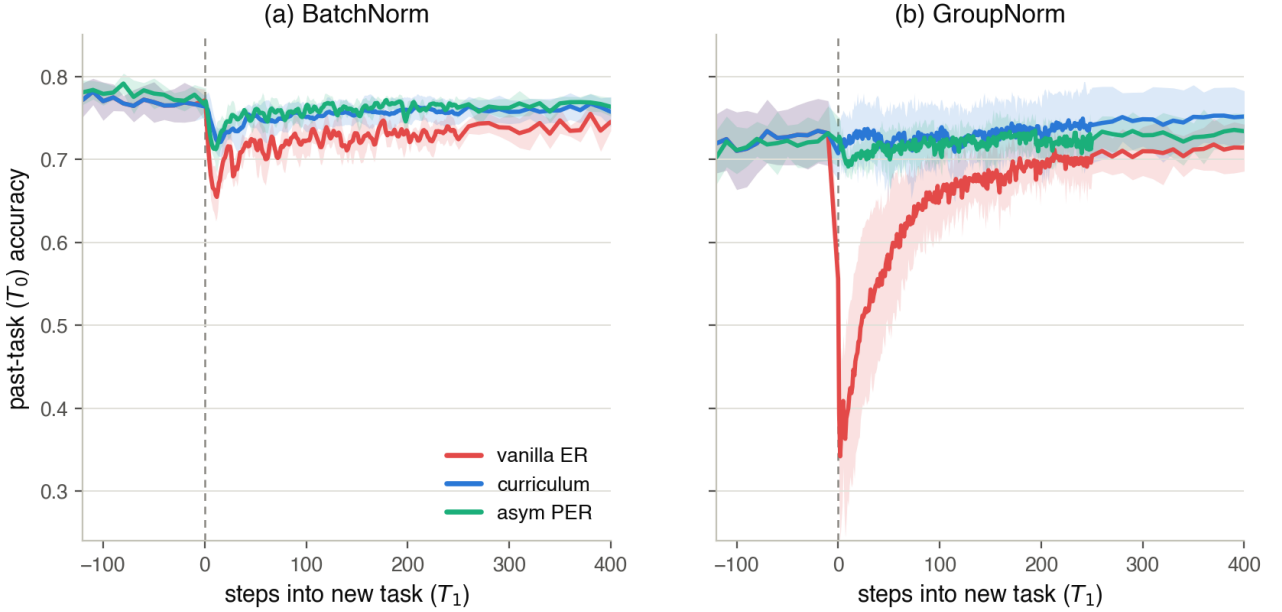


Figure 6.4: Per-step past-task (T_0) accuracy across the CIFAR-10 transition (ResNet-18, momentum 0.9, buffer 5,000), one panel per normalisation, each overlaying vanilla ER (red), the shipping curriculum (blue) and asymmetric PER (aqua); seed mean with $\pm$ std bands, $x=0$ the task switch. (a) BatchNorm: vanilla ER dips and re-adapts slowly while both gates hold T_0 near its plateau. (b) GroupNorm: vanilla ER crashes T_0 almost to chance and crawls back over the whole recovery window, whereas both gates hold it flat through the switch — cutting the deep transient and lifting worst-case retention from ≈ 0.31 to ≈ 0.67 . On this backbone the transient stays open for hundreds of steps, so the area carries as much of the comparison as the depth (Table 6.4, Appendix C.5).

re-inflates under momentum. (iv) Read together, the momentum crosses make the head-to-head comparison regime-dependent rather than a frontier: at momentum 0 and a matched buffer the directional gate is strictly better; with momentum on, required for accuracy, it re-inflates and the scalar gate takes over at vanilla-ER cost. Both gates carry to a ResNet-18 on CIFAR-10 under both normalisations at matched accuracy and much improved worst-case retention; the vanilla and curriculum momentum signatures carry too, with the directional re-inflation confounded by under-convergence on the deep backbone. The discussion reads these findings against the account of Section 3.

7 Discussion

The results defend the two-level account and, with it, the gate view of replay. This section reads them back against the theory: what the two levels explain that a single additive decomposition could not (Section 7.1), how the two gates divide the stability–plasticity work along the feedback/feedforward and scalar/directional axes (Section 7.2), the limits of the evidence (Section 7.3), and where the taxonomy points next (Section 7.4).

7.1 Why the gap needs two levels

The central empirical claim is that the stability gap is not one quantity but two failures of the same optimisation, kept apart because they answer to different interventions. The 2×2 of Section 6.1 is the direct test. Removing the overshoot drivers by construction leaves a deterministic arc (D4: 1.9 pp of depth but a clear bend, area 3.2, with the replay gradient exact and the buffer noiseless); fixing the path with the curriculum leaves the overshoot (C7 at momentum 0: 13.9 pp, essentially all of it); only doing both closes the gap (C8: 0.2 pp). Neither intervention is a weaker version of the other. The arc is a property of which path the descent dynamics define, and no amount of optimiser hygiene, exact gradients, balanced magnitudes, zero noise, touches it; the overshoot is how far the discrete, noisy, accelerated iterate leaves whatever path it is on, and no change of path removes it. This is why the levels must be named separately: an intervention aimed at one is silent on the other.

This supersedes the additive decomposition an earlier version of this work reported. Partitioning the gap into a magnitude share, a variance share, and a trajectory share is order-dependent (a noisy replay gradient inflates the first-step magnitude, so the factors are not separable summands) and maps factors to metrics the data then crosses: the curriculum’s momentum-0 tail looks like a “trajectory” term until C8 removes it with the exact buffer and no momentum at all (Appendix C.2, Table C.3), revealing it as variance. The

two-level split is by kind — which path is followed versus how tightly it is followed — and that is the distinction the interventions respect.

The split also settles what the gap is not the price of: C8 closes the transient to 0.2 pp while its accuracy sits 5.5 pp below vanilla ER (Section 6.1), so the permanent rise to the joint-basin level is the stability–plasticity cost and everything below the settled level and back is failure to follow the monotone reference path, not the cost of learning. This is why the end-referenced area is anchored to the recovered plateau rather than the pre-switch baseline — it attributes the gap to the trajectory rather than to capacity loss.

Finally, the two levels explain why the metric that matters changes with the backbone. On the MLP the arc is cheap: the network reconverges in a few steps, so the open bend integrates to little and the perceived severity is the overshoot spike. On the ResNet the same transient stays open for hundreds of steps (Figure 6.4), so the arc, the part that persists, is where most of the accuracy the gap costs accumulates, and it is where both gates’ margin over vanilla is largest (Table 6.4). Depth and area are not two views of one number; they are the signatures of the two levels, and which one dominates the cost is set by how slowly the network recovers. A depth-only reading would have understated the result on exactly the architectures where continual-evaluation cost is highest.

7.2 Two gates, two ways to divide the work

Written in the unified update of Section 3.3, every replay method gates the new-task push while restoring at full strength, and the curriculum and asymmetric PER are the two pure corners of the taxonomy: a scalar gate driven by feedback, and a directional gate driven by feedforward. The results show the two corners working and failing exactly where the taxonomy says they should, and the single fact that organises the whole comparison is *where* the gate attenuates the push.

The scalar feedback gate attenuates at source. It multiplies the entire new-task loss by a schedule that starts near zero, so early in the transition there is almost no push to discretise, accumulate, or let noise act on, and by the time the weight has grown the trajectory has already settled toward the joint basin. This has three consequences the data confirm. It composes with momentum: because the push is small and clean at source, momentum finds a stochastic residual to average and closes the gap it left untouched under vanilla ER (Section 6.2), while the accumulator has no sustained full-strength direction to rebuild. It is blind-spot-proof: the adaptive schedule reads the live replay-gradient norm, a signal that fires whatever direction the damage arrives from, so it cannot be steered past a threat it did not model. And it pays a

plasticity debt: holding the loss down slows the new task, which is why C8 at momentum 0 is gap-free but 5.5 pp down on accuracy, and why the debt is repaid only by acceleration, momentum lifting C8 to the highest accuracy in the benchmark (Table C.3). The scalar gate’s price is plasticity, and momentum is how it is paid.

The directional feedforward gate attenuates per step. It filters the push through the replay Fisher every step, passing motion where the past task is flat and braking it where the past task is curved, and it never slows the new task at source, so it protects the past task at no plasticity cost, ACC and short-horizon plasticity rising as the filter strengthens (Table 6.3). The same per-step locality is its weakness on both of the taxonomy’s predicted failure axes. Under momentum the protection unravels: the driving magnitude of the new-task gradient is untouched, so velocity accumulated across steps reconstructs displacement in the very directions the filter shrinks one step at a time, and depth re-inflates from ≈ 2 to ≈ 8 pp (Section 6.3). This is the same lever as the curriculum, momentum, producing the opposite outcome, and the difference is entirely one of where the attenuation was applied: source versus step. That single distinction accounts for the whole momentum column of the study, the vanilla depth null (no accumulated velocity in the deterministic first step), the curriculum’s composition (source attenuation), and the directional gate’s re-inflation (step attenuation).

The directional gate’s second failure is the sharper one, because it is the case for feedback. At the strongest filter its curvature model has blind spots: the sampled Fisher rates a direction flat that the true past-task loss punishes further out, new-task pressure escapes through it, and the past-task curve slides off a late cliff (the $\delta=0.03$ full-buffer cliffs, reproduced under a stronger solver, Section 6.3). A feedforward gate can only be as good as its model of the past task, and that model is estimated on finite data under a saturating loss. A feedback gate has no such blind spot, because it does not model the damage in advance, it reads it. The cliffs and the plasticity debt are therefore the two gates’ defining costs, and they are complementary: the feedforward gate is cheap in plasticity but myopic, the feedback gate is blind-spot-proof but slow. This is why the two do not sit on a fixed frontier but flip with momentum (Section 6.4). At a matched memory budget and no momentum the directional gate dominates, because it pays no plasticity and momentum has not yet unwound its filter. Momentum, required for accuracy, then re-inflates the directional gate’s per-step filter while composing with the scalar gate at source, so the scalar gate becomes the lower-gap method in the regime that ships — and does so at vanilla-ER compute against the directional gate’s per-step Fisher solve. The frontier reading of an earlier draft was an artefact of comparing the gates

at mismatched buffer sizes (Section 6.4). That both legs carry to the ResNet at matched accuracy and much improved worst-case retention (Table 6.4), and that the two mechanism signatures not confounded by convergence reproduce there (Appendix C.7, Table C.8), is the evidence that it is the gate principle, not either particular method, that generalises.

7.3 Limitations

Several limits bound these claims. The first is statistical: five seeds floor the two-sided Wilcoxon test at $p = 0.0625$, so the argument rests on effect sizes and seed-paired direction, not a 0.05 threshold the design cannot reach. The effects the story leans on, the order-of-magnitude depth reductions on CIFAR, the C7→C8 variance collapse, the momentum re-inflation of the directional gate, are large relative to the seed spread and unanimous across seeds; the smaller ones, the adjacent- δ steps and the $\lambda_{\min}$ trade-offs, are read as directional consistency only, and one, the adaptive-versus-best-linear depth, does not reach even that ($p = 0.44$).

The second is the accuracy drift of the scalar gate at momentum 0: lengthening the linear ramp lowers depth but costs several points of final accuracy (Table 6.2), so the method’s accuracy-neutrality is a property of the full curriculum-plus-momentum configuration, not of the ramp alone. The third is the directional gate’s cliffs, which we establish at the blind-spot level the solver control supports; the finer softmax-saturation account of why they land on the full-buffer leg is flagged as interpretation, not evidence. The fourth is the CIFAR momentum cross: the directional-gate re-inflation signature does not reproduce on the ResNet, and while we attribute this to momentum-0 under-convergence of the backbone rather than to a failure of the mechanism (which the MLP isolates cleanly), a clean deep-backbone test of that one signature would require a converged momentum-0 baseline we did not run. The fifth is the three-task group-norm sequence, where the adaptive ratio loses conditioning at a boundary between two near-identical corruptions and the schedule underperforms its control with high seed variance; we flag it as a reliability boundary of the gradient-ratio feedback signal, absent from the batch-norm variant, rather than a property of the gate principle.

Two scope limits are worth stating plainly. The full-buffer legs of both the driver block and the δ sweep store the entire past task and so break the limited-memory premise of continual learning; they are diagnostic probes that isolate the arc and split momentum’s roles, and the practical configurations are the 1k-buffer curriculum (C7_M) and the standard-buffer directional gate. And the directional gate’s per-step Fisher-vector solve adds a fixed multiple of the backward cost per step (25 conjugate-gradient iterations

here), a real overhead on large backbones — up to $4\times$ the gate-active wall-clock of the curriculum on the ResNet-18 (Table C.9) — that the scalar gate, a single changing scalar with no stored curvature, does not carry: its cost is indistinguishable from vanilla ER. The headline analysis also rests on a single-transition, two-task design chosen to isolate one boundary; a systematic study across many boundaries, longer sequences, and class-incremental rather than domain-incremental shifts is left open.

7.4 Outlook

The taxonomy that organises the two methods also names what is missing from it. The two gates studied here occupy two of its four corners, scalar $\times$ feedback and directional $\times$ feedforward, and the empty corner, a directional gate driven by a live damage signal rather than by a precomputed curvature model, is exactly the design the cliffs argue for: it would filter the push by direction, keeping the plasticity that makes the feedforward gate cheap, while reading the damage as it arrives, closing the blind spot that makes it myopic. Building and testing such a feedback-directional hybrid is the most direct consequence of these results.

Two narrower directions follow from the failure modes. The directional gate’s momentum re-inflation is a per-step-attenuation artefact; a version that attenuates the accumulated velocity rather than the instantaneous step, or that damps the push at source in the directions the Fisher flags, would be predicted to compose with momentum the way the scalar gate does, a testable claim. And momentum’s gap-closing role was shown to be interchangeable with source variance reduction (C8), which suggests that iterate averaging, a variance reducer that carries none of momentum’s accumulator-driven overshoot, could close the stochastic residual without the small transient price acceleration re-introduces. On the practical side, the regime dependence gives a clean rule. With momentum off the directional gate is strictly better, but momentum is needed for accuracy on any realistic backbone, and there the scalar gate closes the gap at almost no final-accuracy penalty, negligible overhead, and no blind-spot cliffs, while the directional gate re-inflates and carries a per-step Fisher cost. The scalar gate is therefore the default recommendation for systems trained with momentum and queried during training; the directional gate is reserved for the momentum-free regime where it dominates.

Statement on the Use of AI Tools

In accordance with the University of Groningen’s guidelines on the responsible use of generative AI, I disclose that AI assistants were used during the preparation of this thesis. The tools were Google Gemini

Pro (version 3 Pro, then version 3.1 Pro from February 2026) and Anthropic Claude Opus (the 4.x line, versions 4.5 through 4.8) — the releases current across the roughly six-month period, January to July 2026, in which the work was carried out. They functioned as assistive tools under my continuous direction, not as autonomous contributors. Specifically, I used them to:

- brainstorm and stress-test ideas, and as a sounding board to reason through arguments and explanations;
- draft and revise prose, which I subsequently edited for accuracy, precision, and voice;
- assist in writing and refactoring the analysis and experiment code, all of which I reviewed, ran, and tested;
- produce first-pass summaries of the literature, which I verified against the primary sources.

The tools were never given open-ended control: every request specified a concrete, bounded task, and all output was reviewed and, where retained, corrected, understood, and integrated by me. All experimental results, figures, and numerical claims were checked against the underlying run data, and all cited work was read in the original. The research questions, experimental design, interpretation of the results, and the overall argument of the thesis are my own, and I take full responsibility for its content, including any errors.

References

- [1] Rahaf Aljundi, Min Lin, Baptiste Goujaud, and Yoshua Bengio. Gradient based sample selection for online continual learning, 2019. URL <https://arxiv.org/abs/1903.08671>. 3, 4
- [2] Arslan Chaudhry, Marc’Aurelio Ranzato, Marcus Rohrbach, and Mohamed Elhoseiny. Efficient lifelong learning with a-gem, 2019. URL <https://arxiv.org/abs/1812.00420>. 5
- [3] Arslan Chaudhry, Marcus Rohrbach, Mohamed Elhoseiny, Thalaiyasingam Ajanthan, Puneet K. Dokania, Philip H. S. Torr, and Marc’Aurelio Ranzato. On tiny episodic memories in continual learning, 2019. URL <https://arxiv.org/abs/1902.10486>. 4
- [4] Felix Draxler, Kambis Veschgini, Manfred Salmhofer, and Fred A. Hamprecht. Essentially no barriers in neural network energy landscape, 2019. URL <https://arxiv.org/abs/1803.00885>. 5
- [5] Jonathan Frankle, Gintare Karolina Dziugaite, Daniel M. Roy, and Michael Carbin. Linear

- mode connectivity and the lottery ticket hypothesis, 2020. URL <https://arxiv.org/abs/1912.05671>. 5
- [6] Timur Garipov, Pavel Izmailov, Dmitrii Podoprikin, Dmitry Vetrov, and Andrew Gordon Wilson. Loss surfaces, mode connectivity, and fast ensembling of dnns, 2018. URL <https://arxiv.org/abs/1802.10026>. 3, 5
- [7] Yiduo Guo, Jie Fu, Huishuai Zhang, Dongyan Zhao, and Yikang Shen. Efficient continual pre-training by mitigating the stability gap, 2024. URL <https://arxiv.org/abs/2406.14833>. 4
- [8] Yunhui Guo, Mingrui Liu, Tianbao Yang, and Tajana Rosing. Improved schemes for episodic memory-based lifelong learning, 2020. URL <https://arxiv.org/abs/1909.11763>. 5
- [9] Md Yousuf Harun and Christopher Kanan. Overcoming the stability gap in continual learning, 2024. URL <https://arxiv.org/abs/2306.01904>. 4
- [10] Dan Hendrycks and Thomas Dietterich. Benchmarking neural network robustness to common corruptions and perturbations. In *International Conference on Learning Representations (ICLR)*, 2019. URL <https://arxiv.org/abs/1903.12261>. 8, 24
- [11] Timm Hess, Tinne Tuytelaars, and Gido M. van de Ven. Two complementary perspectives to continual learning: Ask not only what to optimize, but also how, 2024. URL <https://arxiv.org/abs/2311.04898>. 3, 4
- [12] Shun ichi Amari. Natural gradient works efficiently in learning. *Neural Computation*, 10(2): 251–276, 1998. 9
- [13] Sandesh Kamath, Albin Soutif-Cormerais, Joost van de Weijer, and Bogdan Raducanu. The expanding scope of the stability gap: Unveiling its presence in joint incremental learning of homogeneous tasks, 2024. URL <https://arxiv.org/abs/2406.05114>. 4
- [14] Ta-Chu Kao, Kristopher T. Jensen, Gido M. van de Ven, Alberto Bernacchia, and Guillaume Hennequin. Natural continual learning: success is a journey, not (just) a destination, 2021. URL <https://arxiv.org/abs/2106.08085>. 3, 4, 5, 6, 7, 25
- [15] James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A. Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, Demis Hassabis, Claudia Clopath, Dharshan Kumaran, and Raia Hadsell. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13):3521–3526, March 2017. ISSN 1091-6490. doi: 10.1073/pnas.1611835114. URL <http://dx.doi.org/10.1073/pnas.1611835114>. 5
- [16] Matthias De Lange, Gido van de Ven, and Tinne Tuytelaars. Continual evaluation for lifelong learning: Identifying the stability gap, 2023. URL <https://arxiv.org/abs/2205.13452>. 3, 4, 6, 8, 26, 27
- [17] David Lopez-Paz and Marc’Aurelio Ranzato. Gradient episodic memory for continual learning, 2022. URL <https://arxiv.org/abs/1706.08840>. 5
- [18] Michael McCloskey and Neil J. Cohen. Catastrophic interference in connectionist networks: The sequential learning problem. *The Psychology of Learning and Motivation*, 24:104–169, 1989. 3
- [19] Martial Mermillod, Aurélie Bugaiska, and Patrick Bonin. The stability-plasticity dilemma: Investigating the continuum from catastrophic forgetting to age-limited learning effects. *Frontiers in psychology*, 4:504, 08 2013. doi: 10.3389/fpsyg.2013.00504. 3
- [20] Seyed Iman Mirzadeh, Mehrdad Farajtabar, Dilan Gorur, Razvan Pascanu, and Hassan Ghasemzadeh. Linear mode connectivity in multitask and continual learning, 2020. URL <https://arxiv.org/abs/2010.04495>. 3, 5
- [21] Roger Ratcliff. Connectionist models of recognition memory: Constraints imposed by learning and forgetting functions. *Psychological Review*, 97:285–308, 04 1990. doi: 10.1037/0033-295X.97.2.285. 3
- [22] Shagun Sodhani, Mojtaba Faramarzi, Saniket Vaibhav Mehta, Pranshu Malviya, Mohamed Abdelsalam, Janarthanan Janarthanan, and Sarath Chandar. An introduction to lifelong supervised learning, 2022. URL <https://arxiv.org/abs/2207.04354>. 3, 5
- [23] Albin Soutif-Cormerais, Antonio Carta, and Joost Van de Weijer. Improving online continual learning performance and stability with temporal ensembles, 2023. URL <https://arxiv.org/abs/2306.16817>. 4
- [24] Edoardo Urettini and Antonio Carta. Online curvature-aware replay: Leveraging 2nd order information for online continual learning, 2025. URL <https://arxiv.org/abs/2502.01866>. 7

- [25] Gido M. van de Ven, Nicholas Soures, and Dhireesha Kudithipudi. *Continual learning and catastrophic forgetting*, page 153–168. Elsevier, 2025. ISBN 9780443157554. doi: 10.1016/b978-0-443-15754-7.00073-0. URL <http://dx.doi.org/10.1016/B978-0-443-15754-7.00073-0>. 4
- [26] Jeffrey S. Vitter. Random sampling with a reservoir. *ACM Trans. Math. Softw.*, 11(1):37–57, March 1985. ISSN 0098-3500. doi: 10.1145/3147.3165. URL <https://doi.org/10.1145/3147.3165>. 4, 24

A Implementation Details

This appendix records the implementation specifics that the methods of Section 5 depend on but that would interrupt the argument in the main text: the experience-replay variants, and the NCL realisation together with the hyperparameters fixed by the rot-MNIST tuning sweep.

A.1 Datasets

rot-MNIST rotates each 28×28 image by the per-task angle and feeds the flattened greyscale vector to the MLP. The CIFAR-10 domain shifts are generated with the `imagecorruptions` package, which implements the standard common-corruptions benchmark of Hendrycks and Dietterich [10]; the Gaussian noise, shot noise, and contrast shifts are applied at severity 3 on the benchmark’s 1–5 scale, per sample at load time, followed by the standard CIFAR-10 channel normalisation. The clean task (“none”) applies the normalisation only.

A.2 Models and Training

Tables A.1 and A.2 give the two backbones, and Table A.3 the training protocol shared across both benchmarks.

Component	Value
Input	784 (flattened 28×28 greyscale)
Hidden layers	2×400 (fully connected, ReLU)
Output	10-class logits
Regularisation	none
Total parameters	478,410

Table A.1: MLP backbone used for the rot-MNIST sweeps.

Component	Value
Input	$3 \times 32 \times 32$ RGB
Stem	3×3 conv, stride 1, no max-pool (CIFAR adaptation)
Stages	4 residual stages, BasicBlocks [2, 2, 2, 2], widths [64, 128, 256, 512]
Normalisation	BatchNorm (default); GroupNorm (min(32, C) groups) variant crossed in Section 5.2.5
Head	global average pool + linear, 10-class logits
Total parameters	≈ 11.2 M

Table A.2: CIFAR-adapted ResNet-18 backbone used for the CIFAR-10 block. The 3×3 stride-1 stem and removed max-pool keep the feature map at 32×32 ; all other details follow the ResNet-18.

Hyperparameter	rot-MNIST (MLP)	CIFAR-10 (ResNet-18)
Epochs per task	1 (online)	10
Batch size	256	256
Optimiser	SGD	SGD
Learning rate	0.1	0.1
Momentum	0.0 or 0.9	0.9 (0.0 in cross)
Weight decay	0.0	0.0
Replay buffer	1,000 or 60,000	5,000
Seeds	5	5 or 3

Table A.3: Training protocol for both benchmarks.

A.3 Experience replay

The replay buffer uses reservoir sampling [26], so the stored set is a uniform sample of the past stream at any budget. Three modes appear in the experiments:

- *Standard ER* (the D1 and all C-series conditions): each step takes one combined backward pass on $L_{\text{current}} + L_{\text{replay}}$, with half the mini-batch drawn from the current task and half sampled with replacement from the buffer.

- *Balanced ER* (D3): two separate backward passes whose gradients are individually unit-normalised before being combined 50/50, removing the magnitude asymmetry between g_{new} and g_{replay} .
- *Full-data replay* (D2, D4): the replay gradient is computed on *every* stored sample exactly once per step (no sub-sampling). Paired with a buffer equal to the full past-task training set (60,000 for rot-MNIST T_0), this yields the deterministic empirical past-task gradient at the current parameters, removing estimator variance.

The λ -curriculum re-weights only the current-task term, minimising $L_\lambda = L_{\text{replay}} + \lambda(t) L_{\text{current}}$; the adaptive schedule reads $\|g_{\text{replay}}\|/\|g_{\text{new}}\|$ through an exponential moving average ($\alpha = 0.05$) and applies the optional floor $\lambda_{\min}$.

A.4 Asymmetric PER

The filtered push $\delta (F_{\text{rep}} + \delta I)^{-1} g_{\text{cur}}$ of Equation (1.2) is computed each step by warm-started conjugate gradients over Fisher-vector products (25 iterations), so the metric is never formed as a matrix; warm-starting from the previous step’s solution keeps the per-step solve cheap across the run. The solver control of Section 5.2.4 reruns the strongest filter ($\delta = 0.03$, full buffer) at 60 iterations to separate properties of the gate from artefacts of an under-converged solve. On the full-buffer leg the replay gradient is exact over all 60k past samples while the Fisher stays estimated on a sampled batch, since a full-buffer Fisher-vector product per CG iteration would dominate the runtime.

A.5 Natural Continual Learning

NCL follows Kao et al. [14] (their Eq. 8 and Algorithm 1). The update preconditions the new-task gradient by the prior covariance Λ_{k-1}^{-1} and adds the Bayesian restoring term,

$$\theta \leftarrow \theta - \eta [\Lambda_{k-1}^{-1} \nabla L_k(\theta) + (\theta - \mu_{k-1})],$$

with Λ_{k-1} summarised by a K-FAC factorisation $F \approx A \otimes G$ per linear layer and an identity prior $p_w = \alpha I$. The trust radius of the paper’s Eq. 7 is absorbed into the learning rate; we log a `tr_scale` diagnostic but do not clip the step.

Tuned hyperparameters. The values used in every NCL condition are those that won the rot-MNIST tuning sweep below (Table A.4). The paper-literal prior $\alpha = 1.0$ over-regularises on rot-MNIST and the damping ε is a numerical safeguard only: the identity prior, not the damping, is what bounds Λ^{-1} . These values were tuned on the two-task rot-MNIST/MLP setting and reused unchanged on the CIFAR-10 ResNet-18, as we did not run a separate NCL sweep on the deeper backbone. NCL’s CIFAR results should therefore be read as those of a method tuned on the smaller benchmark rather than re-optimised for CIFAR, a caveat we revisit in Section 7.3.

Hyperparameter	Symbol	Value
Prior precision	α (<code>prior_init</code>)	0.1
Damping	ε (<code>damping</code>)	1e−3
Fisher samples (K-FAC)	<code>fisher_samples</code>	1,000
Trust radius (diagnostic)	r (<code>trust_radius</code>)	1.0
Learning rate	η	0.1
Momentum	μ	0.9

Table A.4: NCL hyperparameters fixed for all conditions, from the rot-MNIST tuning sweep of Table A.5.

Tuning sweep. The prior precision α was tuned on the two-task rot-MNIST/MLP setting (Table A.5), holding damping at 1e−3 and `fisher_samples` at 1,000. Smaller α improves over the paper-literal $\alpha = 1.0$, but the gain saturates: $\alpha = 0.1$ gives the best stability–accuracy trade-off, while $\alpha \leq 0.03$ becomes unstable at $\eta = 0.1$ (the natural gradient amplifies the raw gradient by up to $1/\alpha$ in any direction), and halving the learning rate does not recover it.

Config	α	η	Momentum
alpha_0.1 (chosen)	0.1	0.1	0.9
alpha_0.03	0.03	0.1	0.9
alpha_0.01	0.01	0.1	0.9
alpha_0.003	0.003	0.1	0.9
alpha_0.03_lowlr	0.03	0.05	0.9

Table A.5: NCL prior-precision tuning sweep on two-task rot-MNIST. Damping ($1e-3$) and `fisher_samples` (1,000) held fixed.

B Metric Definitions

This appendix gives the full definitions of every metric recorded by the evaluation pipeline of Section 5.1.2. The main text explains the measures the argument turns on; the tables below are the look-up reference for the complete set.

Stability-gap metrics. Computed from the dense per-step evaluation. Write a_j^{pre} for the pre-task accuracy on task j and $a_j(t)$ for its accuracy at step t of the new task.

Metric	Definition	Role
gap_depth	$\max_j (a_j^{\text{pre}} - \min_t a_j(t))$ over the full record window	Primary metric, worst single point of instability
gap_area_end	$\sum_j \int_0^T \max(0, b_j - a_j(t)) dt$, trapezoidal over the full new-task record (to task end T), with $b_j = \min(a_j^{\text{pre}}, \bar{a}_j^{\text{end}})$ and $\bar{a}_j^{\text{end}}$ the trailing-window mean (last $\max(5, \lceil 0.1n \rceil)$ samples); accuracy at or above b_j contributes 0	Transient cost reported in the results: dip below the recovered level until it closes, excluding permanent forgetting and continued learning (units: accuracy $\cdot$ steps)
recovery_steps	Given a record has dipped below the threshold, the first step (counted from the transition) at which <i>every</i> past task is simultaneously at $\geq 90\%$ of its pre-task baseline	None if no sub-threshold dip occurs, or if recovery is not reached within the record
gap_area	As gap_area_end but referenced to the pre-task baseline throughout: $\sum_j \int_0^T \max(0, a_j^{\text{pre}} - a_j(t)) dt$	Pre-referenced area; bundles the transient dip with permanent below-baseline drift. Superseded by gap_area_end , logged for back-comparability
max_drop	gap_depth restricted to the first 200 steps	Legacy fixed-window depth, kept for back-comparability

Table B.1: Stability-gap metrics, recorded from dense per-step evaluation. **gap_depth** and **gap_area_end** are the two axes used in the main analysis, how deep the gap cuts and how long it stays open, together with **recovery_steps**; the rows below the rule are secondary or legacy measures logged by the pipeline but not reported in the main text.

Continual-learning metrics. Following Lange et al. [16], computed at task boundaries and over the full record. Let $A(E_j, f_n)$ be the accuracy on the test set of task j evaluated at the model f_n after training stage n , and let N be the number of tasks. The full task-boundary accuracy matrix $R[i, j] = A(E_j, f_{T_i})$ is logged for every run, so any downstream metric can be reconstructed.

Metric	Definition	Role
ACC	$\frac{1}{N} \sum_j A(E_j, f_{T_{N-1}})$	Average task accuracy at the final model
FORG	$\frac{1}{N-1} \sum_{i < N-1} [A(E_i, f_{T_i}) - A(E_i, f_{T_{N-1}})]$	Average drop from right-after-learning to final
min-ACC	$\frac{1}{N-1} \sum_{i < N-1} \min\{A(E_i, f_n) : n > T_i\}$	Worst-case retention since learning
WF _w	Mean over tasks of the largest accuracy drop in any window of w consecutive evaluations ($w \in \{10, 100\}$)	Worst-case stability over short / medium horizons
WP _w	Symmetric counterpart of WF _w : the largest accuracy gain ($w \in \{10, 100\}$)	Plasticity / learning velocity
WC-ACC	$\frac{1}{N} A(E_{N-1}, f_{T_{N-1}}) + (1 - \frac{1}{N}) \text{min-ACC}$	Worst-case trade-off — a lower bound on ACC

Table B.2: Continual-learning metrics of Lange et al. [16], recorded at task boundaries and over the full record.

Gradient-fidelity diagnostics. Recorded at the dense cadence for the decomposition conditions only. The tracker compares the replay-buffer gradient g_{replay} against the true past-task gradient g_{true} , computed at every step by a separate forward and backward pass over the entire past-task training set (no sub-sampling, so g_{true} is the deterministic empirical past-task gradient at the current parameters).

Diagnostic	Logged name	Role
$\cos(g_{\text{replay}}, g_{\text{true}})$	<code>true_grad_cosine</code>	Buffer-side directional fidelity, the estimator-variance driver of Section 5.2.1
$\ g_{\text{replay}}\ / \ g_{\text{true}}\ $	<code>true_grad_mag_ratio</code>	Buffer-side magnitude fidelity
$\ g_{\text{new}}\ / \ g_{\text{replay}}\ $	<code>grad_ratio</code>	Update-side magnitude asymmetry, large at step 1 when $\ g_{\text{replay}}\ \approx 0$

Table B.3: Gradient-fidelity diagnostics. Each is logged as a per-step series together with its mean (and minimum, for the cosine). The first two carry the `true_grad` prefix because they are measured against g_{true} ; `grad_ratio` compares g_{replay} to the current-task gradient g_{new} and its inverse drives the adaptive curriculum of Section 5.2.2.

C Complete Results

This appendix gives the full metric set for every condition, as recorded by the aggregation pipeline of Section 5.1.2. Each row is the mean over five seeds (three for the CIFAR generalisation block) with the seed standard deviation. Accuracy-type metrics (ACC, FORG, min-ACC, WF₁₀₀, WP₁₀₀, WC-ACC) are fractions; `gap_depth` is in percentage points (pp); `gap_area_end` is the transient cost (accuracy-fraction $\times$ steps, Table B.1), the dip below the level each task recovers to, so permanent forgetting does not contribute; “Recov.” is `recovery_steps`, with “—” marking conditions where every past task stayed above the 90% recovery threshold for the whole window (no recovery event to time).

C.1 rot-MNIST: Decomposition (D-series)

Condition	ACC	FORG	min-ACC	WF ₁₀₀	WP ₁₀₀	WC-ACC	Depth	Area end	Recov.
<i>Momentum 0.0</i>									
D1 vanilla	0.873 $\pm$ 0.005	0.019 $\pm$ 0.020	0.706 $\pm$ 0.056	0.147 $\pm$ 0.029	0.736 $\pm$ 0.020	0.794 $\pm$ 0.028	19.5 $\pm$ 4.1	6.3 $\pm$ 0.5	3.6 $\pm$ 0.9
D2 fulldata	0.890 $\pm$ 0.009	-0.016 $\pm$ 0.009	0.719 $\pm$ 0.039	0.126 $\pm$ 0.020	0.736 $\pm$ 0.018	0.801 $\pm$ 0.022	18.3 $\pm$ 2.5	8.1 $\pm$ 0.7	3.0 $\pm$ 1.2
D3 balanced	0.849 $\pm$ 0.005	0.031 $\pm$ 0.018	0.824 $\pm$ 0.006	0.045 $\pm$ 0.002	0.721 $\pm$ 0.004	0.836 $\pm$ 0.004	5.7 $\pm$ 1.9	1.4 $\pm$ 0.4	—
D4 fulldata balanced	0.869 $\pm$ 0.002	-0.014 $\pm$ 0.013	0.862 $\pm$ 0.002	0.030 $\pm$ 0.007	0.710 $\pm$ 0.003	0.852 $\pm$ 0.002	1.9 $\pm$ 1.3	3.2 $\pm$ 0.5	—
D7 NCL reference	0.770 $\pm$ 0.012	0.057 $\pm$ 0.013	0.809 $\pm$ 0.022	0.109 $\pm$ 0.023	0.639 $\pm$ 0.012	0.762 $\pm$ 0.011	7.2 $\pm$ 1.0	0.3 $\pm$ 0.1	125.0
<i>Momentum 0.9</i>									
D1 vanilla	0.928 $\pm$ 0.003	0.058 $\pm$ 0.007	0.797 $\pm$ 0.028	0.092 $\pm$ 0.016	0.808 $\pm$ 0.012	0.878 $\pm$ 0.014	15.9 $\pm$ 2.6	1.0 $\pm$ 0.2	12.6 $\pm$ 1.7
D2 fulldata	0.964 $\pm$ 0.002	-0.010 $\pm$ 0.003	0.807 $\pm$ 0.009	0.082 $\pm$ 0.004	0.808 $\pm$ 0.012	0.885 $\pm$ 0.004	14.9 $\pm$ 0.9	3.1 $\pm$ 0.2	10.6 $\pm$ 1.1
D3 balanced	0.932 $\pm$ 0.005	0.038 $\pm$ 0.003	0.897 $\pm$ 0.006	0.040 $\pm$ 0.003	0.822 $\pm$ 0.011	0.922 $\pm$ 0.003	5.9 $\pm$ 0.6	0.4 $\pm$ 0.2	—
D4 fulldata balanced	0.967 $\pm$ 0.001	-0.022 $\pm$ 0.003	0.942 $\pm$ 0.005	0.016 $\pm$ 0.004	0.834 $\pm$ 0.010	0.950 $\pm$ 0.002	1.3 $\pm$ 0.4	0.2 $\pm$ 0.1	—
D7 NCL reference	0.810 $\pm$ 0.019	0.021 $\pm$ 0.004	0.871 $\pm$ 0.012	0.218 $\pm$ 0.014	0.686 $\pm$ 0.012	0.778 $\pm$ 0.022	8.5 $\pm$ 1.1	0.6 $\pm$ 0.1	5.0

Table C.1: rot-MNIST decomposition block, full metrics, at both momentum settings. Buffer-fidelity diagnostics for these runs are in Table C.4.

C.2 rot-MNIST: λ -Curriculum (C-series)

Condition	ACC	FORG	min-ACC	WF ₁₀₀	WP ₁₀₀	WC-ACC	Depth	Area end	Recov.
<i>Momentum 0.0</i>									
C1 linear N50	0.851 $\pm$ 0.023	0.042 $\pm$ 0.021	0.707 $\pm$ 0.019	0.148 $\pm$ 0.011	0.748 $\pm$ 0.006	0.784 $\pm$ 0.021	17.5 $\pm$ 1.7	4.9 $\pm$ 0.5	35.8 $\pm$ 6.3
C2 linear N100	0.844 $\pm$ 0.025	0.046 $\pm$ 0.023	0.719 $\pm$ 0.033	0.140 $\pm$ 0.025	0.730 $\pm$ 0.006	0.786 $\pm$ 0.019	16.2 $\pm$ 4.3	3.6 $\pm$ 0.3	64.0 $\pm$ 8.6
C3 linear N200	0.828 $\pm$ 0.025	0.054 $\pm$ 0.024	0.739 $\pm$ 0.048	0.128 $\pm$ 0.043	0.688 $\pm$ 0.004	0.784 $\pm$ 0.034	14.3 $\pm$ 4.6	1.8 $\pm$ 0.6	132.8 $\pm$ 43.4
C4 adaptive	0.833 $\pm$ 0.025	0.050 $\pm$ 0.019	0.754 $\pm$ 0.025	0.111 $\pm$ 0.016	0.703 $\pm$ 0.007	0.794 $\pm$ 0.020	12.8 $\pm$ 2.9	2.2 $\pm$ 0.5	88.8 $\pm$ 6.1
C5 adaptive lmin0.05	0.835 $\pm$ 0.025	0.048 $\pm$ 0.019	0.749 $\pm$ 0.027	0.113 $\pm$ 0.016	0.702 $\pm$ 0.006	0.793 $\pm$ 0.020	13.2 $\pm$ 3.2	2.2 $\pm$ 0.5	88.8 $\pm$ 6.1
C6 adaptive lmin0.10	0.836 $\pm$ 0.026	0.048 $\pm$ 0.021	0.743 $\pm$ 0.026	0.120 $\pm$ 0.019	0.700 $\pm$ 0.007	0.791 $\pm$ 0.019	13.9 $\pm$ 3.3	2.4 $\pm$ 0.5	88.4 $\pm$ 11.0
C7 adaptive lmin0.20	0.837 $\pm$ 0.031	0.049 $\pm$ 0.024	0.743 $\pm$ 0.030	0.119 $\pm$ 0.023	0.710 $\pm$ 0.010	0.792 $\pm$ 0.024	13.9 $\pm$ 3.4	2.7 $\pm$ 0.6	64.8 $\pm$ 8.2
<i>Momentum 0.9</i>									
C1 linear N50	0.932 $\pm$ 0.003	0.053 $\pm$ 0.005	0.892 $\pm$ 0.004	0.041 $\pm$ 0.004	0.830 $\pm$ 0.012	0.927 $\pm$ 0.003	6.4 $\pm$ 0.5	0.2 $\pm$ 0.1	—
C2 linear N100	0.932 $\pm$ 0.003	0.051 $\pm$ 0.008	0.892 $\pm$ 0.005	0.036 $\pm$ 0.003	0.826 $\pm$ 0.012	0.926 $\pm$ 0.002	6.4 $\pm$ 0.6	0.2 $\pm$ 0.1	—
C3 linear N200	0.925 $\pm$ 0.007	0.051 $\pm$ 0.006	0.893 $\pm$ 0.004	0.031 $\pm$ 0.004	0.819 $\pm$ 0.010	0.920 $\pm$ 0.006	6.2 $\pm$ 0.6	0.1 $\pm$ 0.0	—
C4 adaptive	0.917 $\pm$ 0.001	0.019 $\pm$ 0.004	0.931 $\pm$ 0.005	0.016 $\pm$ 0.003	0.798 $\pm$ 0.010	0.915 $\pm$ 0.001	2.5 $\pm$ 0.5	0.3 $\pm$ 0.1	—
C5 adaptive lmin0.05	0.922 $\pm$ 0.001	0.023 $\pm$ 0.004	0.930 $\pm$ 0.005	0.016 $\pm$ 0.002	0.802 $\pm$ 0.009	0.921 $\pm$ 0.001	2.6 $\pm$ 0.5	0.1 $\pm$ 0.0	—
C6 adaptive lmin0.10	0.927 $\pm$ 0.002	0.028 $\pm$ 0.005	0.926 $\pm$ 0.004	0.017 $\pm$ 0.003	0.806 $\pm$ 0.009	0.927 $\pm$ 0.002	3.0 $\pm$ 0.4	0.0 $\pm$ 0.0	—
C7 adaptive lmin0.20	0.931 $\pm$ 0.002	0.035 $\pm$ 0.005	0.919 $\pm$ 0.003	0.021 $\pm$ 0.004	0.812 $\pm$ 0.009	0.930 $\pm$ 0.001	3.7 $\pm$ 0.4	0.0 $\pm$ 0.0	—

Table C.2: rot-MNIST λ -curriculum sweep, full metrics, at both momentum settings. All conditions applied on top of standard ER (1 k reservoir).

Full-buffer momentum diagnostic (C8). Table C.3 crosses the shipping curriculum with the full 60k buffer at both momentum settings, isolating momentum’s two roles (Section 6.2). Source variance reduction alone (C8, full buffer, $\mu=0$) already closes the gap to 0.2 pp, so the residual the curriculum leaves at $\mu=0$ is estimator variance, not structure; momentum’s gap-closing role is interchangeable with the full buffer. Acceleration alone (the $\mu=0.9$ column) restores accuracy: C8_M reaches 0.955, the highest in the benchmark, while re-introducing only a 2.0 pp transient. C8_M’s tail (area 0.38) exceeds C7_M’s (0.03) because C7_M’s residual is a permanent step to a lower plateau that the end-referenced area excludes (Section 5.1.2), whereas C8_M recovers to the highest accuracy in the benchmark, so its dip is a genuine transient the area counts in full. C8/C8_M are diagnostic probes: the full buffer defeats the limited-memory premise, and the practical scalar-gate configuration remains C7_M (3.7 pp / 0.03 / 0.931) at 1/60th the memory.

	C7 (1k, $\mu=0$)	C7 _M (1k, $\mu=0.9$)	C8 (full, $\mu=0$)	C8 _M (full, $\mu=0.9$)
gap_depth (pp)	13.9	3.7	0.2	2.0
gap_area_end	2.7	0.03	0.05	0.38
ACC	0.837	0.931	0.818	0.955

Table C.3: Curriculum $\times$ full buffer $\times$ momentum on rot-MNIST, isolating momentum’s two roles. Source variance reduction alone (C8) closes the gap; acceleration alone (the μ column) restores accuracy. The residual at C7 ($\mu=0$) is variance, not structure.

C.3 rot-MNIST: Buffer-Fidelity Diagnostics

The gradient diagnostics of Appendix B were logged for the decomposition block only. The full-data conditions (D2, D4) reach exact fidelity by construction, the replay gradient equals the deterministic empirical past-task gradient, so cosine and magnitude ratio are 1.000. The finite-buffer conditions (D1, D3) show the directional and magnitude error a 1,000-sample reservoir incurs against g_{true} .

Condition	cōs	cos _{min}	$\bar{\rho}$
D1 vanilla ($\mu=0$)	0.711 $\pm$ 0.035	0.086 $\pm$ 0.103	1.040 $\pm$ 0.039
D3 balanced ($\mu=0$)	0.665 $\pm$ 0.043	0.025 $\pm$ 0.140	1.137 $\pm$ 0.060
D1 vanilla ($\mu=0.9$)	0.551 $\pm$ 0.033	0.135 $\pm$ 0.069	0.418 $\pm$ 0.014
D3 balanced ($\mu=0.9$)	0.441 $\pm$ 0.039	-0.052 $\pm$ 0.117	0.265 $\pm$ 0.024
D2, D4 (full data)	1.000	1.000	1.000

Table C.4: Buffer-fidelity diagnostics for the decomposition block: cōs and cos_{min} are the mean and minimum of $\cos(g_{\text{replay}}, g_{\text{true}})$ over the window; $\bar{\rho}$ is the mean of $\|g_{\text{replay}}\|/\|g_{\text{true}}\|$.

C.4 rot-MNIST: Symmetric vs. Asymmetric Preconditioning

Leaving the restoring gradient g_{rep} at full strength is the directional gate’s defining choice (Section 6.3). The alternative is to precondition the summed update, $d = \delta(F_{\text{rep}} + \delta I)^{-1}(g_{\text{cur}} + g_{\text{rep}})$, the symmetric variant that

passes both gradients through the same metric. Because g_{rep} is the gradient of the replay loss, it lies in the top eigenspace of that loss’s own Fisher F_{rep} , so the symmetric filter shrinks the restoring gradient *harder* than the harmful push it is meant to brake and cannot preferentially protect the past task. Table C.5 sweeps the same δ on both variants, and they are mirror images. As the filter strengthens ($\delta \rightarrow 0$) the asymmetric gate’s forgetting falls ($2.7 \rightarrow 1.2$ pp) and its accuracy rises, while the symmetric gate’s forgetting *climbs* ($3.0 \rightarrow 6.6$ pp) and its accuracy falls; every one of these four trends holds across all five seeds. Symmetric depth barely moves across the sweep ($5.7 \rightarrow 6.7$ pp) and at $\delta=1.0$ even sits below the asymmetric gate’s (5.7 vs. 9.0 pp), because filtering the whole update also damps the per-step *expression* of the dip. This is the area caveat of Section 5.1.2 made concrete: the symmetric gate’s end-referenced area collapses toward zero ($2.7 \rightarrow 0.01$) not because the dip recovers but because it stops recovering, settling into a permanent accommodation that the area excludes and the forgetting metric records. Read on depth or transient area the symmetric gate looks competitive; read on forgetting and final accuracy it degrades monotonically as its filter strengthens. Asymmetry is what turns the filter from a reshuffling of the damage into a net removal of it.

δ	Asymmetric (g_{rep} free)			Symmetric (joint)		
	depth (pp)	FORG (pp)	ACC	depth (pp)	FORG (pp)	ACC
1.0	9.0	2.7	0.869	5.7	3.0	0.872
0.3	7.4	2.3	0.873	5.9	4.2	0.867
0.1	5.9	1.8	0.876	6.2	5.7	0.861
0.03	4.8	1.2	0.880	6.7	6.6	0.857

Table C.5: Asymmetric versus symmetric preconditioning on the standard 1k buffer at momentum 0, five seeds, sweeping the shared damping δ . Both variants filter through the replay Fisher F_{rep} ; they differ only in whether the metric is applied to the new-task push alone (asymmetric, g_{rep} left free) or to the summed update (symmetric). As $\delta \rightarrow 0$ the asymmetric gate improves on every axis while the symmetric gate degrades on every axis; symmetric depth stays flat because the filter suppresses the per-step expression of the dip even as the underlying forgetting grows (Section 5.1.2).

C.5 CIFAR-10: Headline Block

Condition	ACC	FORG	min-ACC	WF ₁₀₀	WP ₁₀₀	WC-ACC	Depth	Area end
G1 vanilla BN	0.723 $\pm$ 0.008	0.010 $\pm$ 0.019	0.644 $\pm$ 0.023	0.127 $\pm$ 0.015	0.484 $\pm$ 0.020	0.667 $\pm$ 0.016	11.8 $\pm$ 2.5	17.5 $\pm$ 4.6
G2 NCL BN	0.710 $\pm$ 0.005	0.064 $\pm$ 0.026	0.504 $\pm$ 0.045	0.167 $\pm$ 0.031	0.513 $\pm$ 0.018	0.611 $\pm$ 0.020	25.8 $\pm$ 5.2	55.1 $\pm$ 11.2
G3 adaptive BN	0.722 $\pm$ 0.011	0.006 $\pm$ 0.016	0.713 $\pm$ 0.017	0.070 $\pm$ 0.010	0.469 $\pm$ 0.016	0.698 $\pm$ 0.013	4.8 $\pm$ 0.7	3.6 $\pm$ 2.2
G4 vanilla GN	0.730 $\pm$ 0.011	-0.023 $\pm$ 0.030	0.309 $\pm$ 0.111	0.294 $\pm$ 0.097	0.415 $\pm$ 0.058	0.507 $\pm$ 0.054	41.2 $\pm$ 13.5	34.9 $\pm$ 23.7
G5 NCL GN	0.711 $\pm$ 0.017	0.012 $\pm$ 0.028	0.423 $\pm$ 0.083	0.200 $\pm$ 0.062	0.391 $\pm$ 0.055	0.563 $\pm$ 0.038	29.7 $\pm$ 10.0	52.6 $\pm$ 13.3
G6 adaptive GN	0.731 $\pm$ 0.020	-0.026 $\pm$ 0.026	0.670 $\pm$ 0.055	0.095 $\pm$ 0.033	0.340 $\pm$ 0.052	0.687 $\pm$ 0.037	5.2 $\pm$ 2.1	1.7 $\pm$ 0.9

Table C.6: CIFAR-10 headline block, full metrics (ResNet-18, momentum 0.9, buffer 5,000, five seeds). “adaptive” is the shipped curriculum with $\lambda_{\text{min}} = 0.20$.

C.6 CIFAR-10: Generalisation Block

Each shift pairs the shipped curriculum (“ship”: adaptive $\lambda_{\text{min}} = 0.20$, buffer 5,000) against a vanilla-ER control on the same shift, at the indicated normalisation, over three seeds.

Condition	ACC	FORG	min-ACC	WC-ACC	Depth	Area end
<i>BatchNorm</i>						
shot noise — vanilla	0.724 $\pm$ 0.011	0.017 $\pm$ 0.010	0.633 $\pm$ 0.017	0.663 $\pm$ 0.014	13.1 $\pm$ 3.0	21.2 $\pm$ 3.9
shot noise — ship	0.723 $\pm$ 0.014	0.010 $\pm$ 0.010	0.708 $\pm$ 0.032	0.697 $\pm$ 0.016	5.6 $\pm$ 1.0	5.8 $\pm$ 1.5
contrast — vanilla	0.671 $\pm$ 0.026	-0.008 $\pm$ 0.006	0.610 $\pm$ 0.063	0.586 $\pm$ 0.048	15.4 $\pm$ 3.9	33.4 $\pm$ 4.5
contrast — ship	0.671 $\pm$ 0.044	-0.012 $\pm$ 0.007	0.727 $\pm$ 0.034	0.643 $\pm$ 0.048	3.7 $\pm$ 1.1	2.2 $\pm$ 0.9
3-task — vanilla	0.740 $\pm$ 0.006	-0.011 $\pm$ 0.013	0.670 $\pm$ 0.015	0.691 $\pm$ 0.010	3.3 $\pm$ 0.4	12.0 $\pm$ 2.4
3-task — ship	0.745 $\pm$ 0.010	-0.021 $\pm$ 0.008	0.677 $\pm$ 0.025	0.696 $\pm$ 0.017	6.9 $\pm$ 2.7	4.1 $\pm$ 1.1
<i>GroupNorm</i>						
shot noise — vanilla	0.718 $\pm$ 0.021	-0.036 $\pm$ 0.036	0.244 $\pm$ 0.075	0.469 $\pm$ 0.032	46.3 $\pm$ 12.0	54.4 $\pm$ 40.8
shot noise — ship	0.705 $\pm$ 0.061	-0.028 $\pm$ 0.012	0.638 $\pm$ 0.108	0.656 $\pm$ 0.087	7.3 $\pm$ 5.9	2.4 $\pm$ 1.1
contrast — vanilla	0.777 $\pm$ 0.022	-0.067 $\pm$ 0.021	0.641 $\pm$ 0.061	0.710 $\pm$ 0.040	8.4 $\pm$ 4.4	3.3 $\pm$ 0.6
contrast — ship	0.778 $\pm$ 0.032	-0.070 $\pm$ 0.013	0.668 $\pm$ 0.077	0.724 $\pm$ 0.054	3.9 $\pm$ 3.1	1.2 $\pm$ 0.8
3-task — vanilla	0.722 $\pm$ 0.020	-0.023 $\pm$ 0.015	0.468 $\pm$ 0.043	0.550 $\pm$ 0.023	4.4 $\pm$ 0.4	24.8 $\pm$ 11.8
3-task — ship	0.721 $\pm$ 0.019	-0.025 $\pm$ 0.020	0.421 $\pm$ 0.114	0.517 $\pm$ 0.070	32.8 $\pm$ 14.5	16.9 $\pm$ 10.2

Table C.7: CIFAR-10 generalisation block, full metrics. The two-task novel corruptions (shot noise, contrast) favour the curriculum under both normalisations; the three-task group-norm sequence is the outlier discussed in Section 6.5.

C.7 CIFAR-10: Momentum Cross and Compute Cost

This subsection collects the three deep-backbone checks summarised in Section 6.5: the momentum cross (Table C.8), the generalisation block (Table C.7 above), and the compute cost (Table C.9).

Momentum cross. The cross repeats the composability test of Section 5.2.3 on the ResNet, pairing momentum-0 runs at group norm against the momentum-0.9 rows. Two of the three predicted signatures reproduce: momentum leaves vanilla ER’s depth essentially unchanged ($40.5 \rightarrow 41.2$ pp, the deterministic-first-step null), and it composes with the curriculum, taking depth from 28.6 to 5.2 pp (source attenuation). The third, momentum re-inflating the directional gate, does not appear: momentum takes asymmetric PER from 22.6 to 5.6 pp instead. We read this as a convergence confound rather than a failure of the account: at momentum 0 the ResNet under-converges the past task in ten epochs (vanilla min-ACC 0.11, ACC 0.65), so the momentum-0 depths sit on a degraded baseline and the directional-gate contrast is dominated by the general convergence gain rather than the per-step-attenuation effect the full-buffer MLP isolates cleanly (Section 6.3). On this backbone we confirm the mechanism only for the two gates whose momentum contrast is not swamped by convergence.

Method (GroupNorm)	depth, $\mu=0$	depth, $\mu=0.9$
Vanilla ER	40.5 ± 4.0	41.2 ± 13.5
Curriculum ($\lambda_{\min}=0.20$)	28.6 ± 8.8	5.2 ± 2.1
Asym. PER ($\delta=0.1$)	22.6 ± 9.7	5.6 ± 1.1

Table C.8: CIFAR-10 momentum cross (group norm, buffer 5,000, five seeds), gap depth in pp. The vanilla null and the curriculum composition reproduce; the directional-gate re-inflation does not, confounded by momentum-0 under-convergence of the ResNet.

Compute cost. The scalar gate is a single changing scalar — one gradient-norm ratio per step, no stored curvature — so its wall-clock is indistinguishable from vanilla ER: on the ResNet-18 the gate-active task (T_1) takes 2,833s under the curriculum against 2,836s under vanilla at batch norm, and 4,749 vs. 4,624s at group norm. The directional gate runs 25 conjugate-gradient Fisher-vector products per step, each on the order of a backward pass, so the same task takes 11,712s (batch norm) and 10,057s (group norm), a $4.1\times$ and $2.1\times$ multiple of the curriculum. As a machine-independent ratio — the slowdown of the gate-active task relative to the un-gated first task — vanilla and the curriculum both sit at $\approx 5\times$ while asymmetric PER sits at $24\text{--}29\times$. On the MLP the same overhead is only $\approx 8\%$ (curriculum 231s, PER 250s), because the cheap backward pass and per-step evaluation dominate the wall clock and mask the solve; the gap widens with backbone depth.

Gate (task T_1 , gate active)	rot-MNIST MLP	ResNet-18 BN	ResNet-18 GN
Vanilla ER	—	2,836 s	4,624 s
Curriculum (scalar)	231 s	2,833 s	4,749 s
Asym. PER ($\delta=0.1$)	250 s	11,712 s	10,057 s

Table C.9: Wall-clock of the gate-active task (T_1), mean over five seeds. The scalar gate’s cost is indistinguishable from vanilla ER; the directional gate’s per-step conjugate-gradient solve multiplies it up to $4\times$ on the ResNet-18, and the multiple grows with backbone depth. The MLP vanilla cell is omitted because its driver-block run additionally carries the per-step gradient-fidelity diagnostic; the curriculum and PER MLP runs carry no such diagnostic and are directly comparable.

C.8 Long-Sequence rot-MNIST (L-series)

Condition	ACC	FORG	min-ACC	WF ₁₀₀	WP ₁₀₀	WC-ACC	Depth	Area_end
<i>Momentum 0.0</i>								
L1 vanilla ER	0.861 ± 0.004	0.060 ± 0.004	0.670 ± 0.055	0.207 ± 0.053	0.443 ± 0.011	0.722 ± 0.043	21.0 ± 5.8	3.14 ± 0.77
L2 curriculum $\lambda_{\min}=0.20$	0.860 ± 0.002	0.031 ± 0.005	0.828 ± 0.007	0.073 ± 0.005	0.400 ± 0.003	0.840 ± 0.005	4.1 ± 0.8	1.62 ± 0.42
L3 asym. PER $\delta=0.1$	0.870 ± 0.003	0.053 ± 0.006	0.843 ± 0.008	0.038 ± 0.007	0.381 ± 0.003	0.862 ± 0.007	4.7 ± 1.2	0.38 ± 0.24
<i>Momentum 0.9</i>								
L1 vanilla ER	0.889 ± 0.007	0.096 ± 0.008	0.845 ± 0.007	0.065 ± 0.008	0.317 ± 0.006	0.870 ± 0.006	8.9 ± 1.7	1.68 ± 0.69
L2 curriculum $\lambda_{\min}=0.20$	0.897 ± 0.002	0.071 ± 0.003	0.871 ± 0.005	0.041 ± 0.004	0.364 ± 0.005	0.888 ± 0.004	5.4 ± 0.6	0.60 ± 0.14
L3 asym. PER $\delta=0.1$	0.884 ± 0.004	0.104 ± 0.004	0.849 ± 0.006	0.054 ± 0.004	0.325 ± 0.005	0.873 ± 0.005	7.5 ± 1.5	0.80 ± 0.63

Table C.10: Five-task rot-MNIST (rotations $[0, 30, 60, 90, 120]^\circ$, four consecutive transitions, five seeds), the refactored L-series: the two shipping gates against vanilla ER, at both momentum settings. Depth is the worst single-boundary drop (gap_depth) and area the end-referenced transient (gap_area_end), aggregated as in the two-task blocks so the numbers are directly comparable. At momentum 0 both gates cut the depth from 21 pp to ≈ 4 pp; under momentum, where the per-boundary drops are already smaller, the curriculum stays ahead on cumulative retention (lowest FORG, highest ACC and WC-ACC) while asymmetric PER holds the smallest per-boundary area but, its per-step filter re-inflated by momentum (Section 6.3), gives up its retention edge and tracks vanilla on FORG.

C.9 Supplementary Figures

This subsection collects supplementary figures: the accuracy–depth scatter of every rot-MNIST gate condition (Figure C.1), the five-task rot-MNIST sequence (the multi-transition check reported in Table C.10), and the three-task group-norm generalisation outlier discussed in Section 6.5.

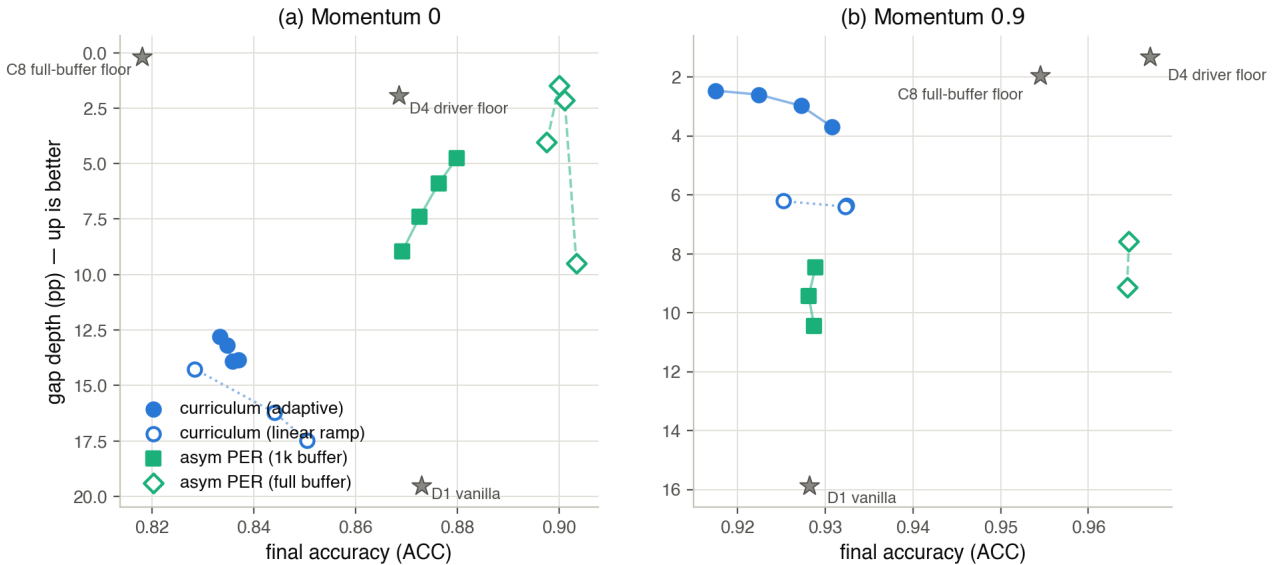


Figure C.1: Accuracy–depth scatter of every rot-MNIST gate condition: final accuracy (right is better) against gap depth (up is better, axis inverted). The λ -path (blue) is the curriculum, adaptive+ $\lambda_{\min}$ as filled circles and linear ramps as open circles; the δ -path (aqua) is asymmetric PER, the 1k buffer as filled squares and the full buffer as open diamonds, each sweeping $\delta = 1.0 \rightarrow 0.03$; grey stars are the anchors (no-gate D1, driver floor D4, full-buffer floor C8). (a) Momentum 0: at a matched 1k buffer the δ -path dominates the λ -path, lower depth *and* higher accuracy, reaching the D4 floor. (b) Momentum 0.9: the apparent trade in this panel is a buffer mismatch — the directional gate’s high-accuracy points (≈ 0.965) are its full 60k-buffer (open-diamond) legs, set against the curriculum’s practical 1k buffer. Held at a common buffer the panel collapses to the regime dependence of Section 6.4: momentum re-inflates the directional gate and the scalar gate becomes the lower-gap method. This figure is retained as the underlying scatter; the frontier reading it once suggested is superseded.

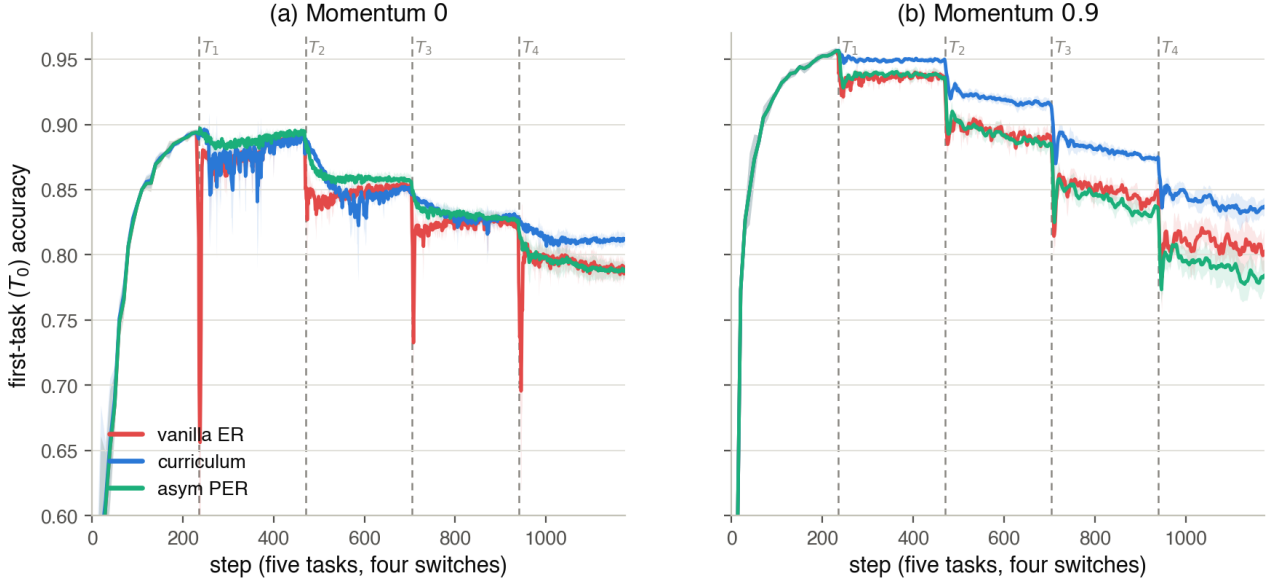


Figure C.2: Five-task rot-MNIST (rotations $[0, 30, 60, 90, 120]^\circ$, four consecutive switches, five seeds with $\pm$ std bands), the refactored L-series: first-task T_0 accuracy across the whole sequence for vanilla ER (red), the shipping curriculum $\lambda_{\min}=0.20$ (blue) and asymmetric PER $\delta=0.1$ (aqua); dashed lines mark the four switches, and the two panels are the two momentum settings. (a) Momentum 0: both gates hold T_0 almost flat through every boundary — asymmetric PER the smoothest of all, with only shallow dips (smallest per-boundary area, Table C.10) — while vanilla ER takes a sharp instantaneous drop at each switch. This is the near-gap-free directional-gate regime the two-task sweep also finds at momentum 0. (b) Momentum 0.9: momentum re-inflates PER’s per-step filter, exactly as on the two-task benchmark (Section 6.3), so its boundary transients return and it tracks vanilla on cumulative retention; the curriculum keeps the boundaries smoothest and holds the highest plateau, the least cumulative forgetting on the long horizon. The single-transition reading thus carries past one boundary, and the momentum roles carry with it.

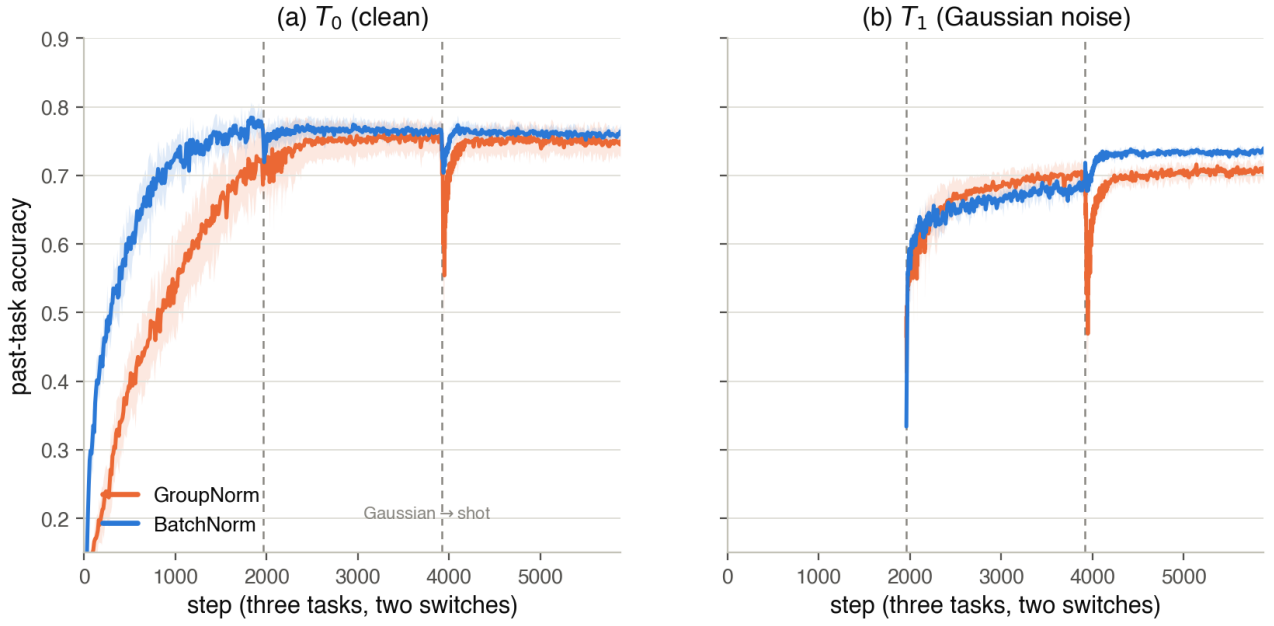


Figure C.3: Per-step past-task accuracy across the three-task CIFAR-10 generalisation sequence (none $\rightarrow$ Gaussian $\rightarrow$ shot, ResNet-18, shipping curriculum, three seeds with $\pm$ std bands), comparing the group-norm run (orange) against its batch-norm counterpart (blue). The two switches (dashed) fall at $\sim 2k$ and $\sim 4k$ steps. The first boundary (none $\rightarrow$ Gaussian) barely perturbs the past tasks in either norm. The second (Gaussian $\rightarrow$ shot, two near-identical corruptions) drives a deep transient in T_0 (a) and T_1 (b) under group norm only, while the batch-norm run holds flat through both switches. The dip is thus localised entirely to the second transition and to group norm, the adaptive-ratio reliability boundary discussed in Section 6.5.

D Detailed Step-by-Step Derivation of the Trajectory Bound

The Mathematical Setup & Definitions

1. The Combined Loss. The total loss function at any point $\lambda \in [0, 1]$ in the curriculum is defined as:

$$L_\lambda(\theta) = L_{\text{replay}}(\theta) + \lambda \cdot L_{\text{current}}(\theta) \quad (\text{D.1})$$

2. Defining the Hessians (∇^2). When we take the derivative of a gradient (∇), we obtain the second derivative matrix, called the Hessian (H).

- $H_{\text{replay}} = \nabla^2 L_{\text{replay}}(\theta)$ (The curvature of the old task)
- $H_{\text{current}} = \nabla^2 L_{\text{current}}(\theta)$ (The curvature of the new task)

Therefore, the Hessian of our combined loss, H_λ , is defined by applying the second derivative operator to the total loss function:

$$\begin{aligned} H_\lambda &= \nabla^2 [L_{\text{replay}}(\theta) + \lambda \cdot L_{\text{current}}(\theta)] \\ H_\lambda &= \nabla^2 L_{\text{replay}}(\theta) + \lambda \cdot \nabla^2 L_{\text{current}}(\theta) \\ H_\lambda &= H_{\text{replay}} + \lambda \cdot H_{\text{current}} \end{aligned}$$

3. Assumption: Local-Minimum along the Interpolation Branch. We assume that the loss functions L_{replay} and L_{current} are twice continuously differentiable (C^2) in a neighbourhood of the interpolation path. Furthermore, we assume there exists a continuously differentiable (C^1) branch of critical points $\lambda \mapsto \Theta^*(\lambda)$ for the interpolated loss L_λ , initialized at $\Theta^*(0) = \theta_0^*$, such that the Hessian remains strictly positive-definite: $H_\lambda(\Theta^*(\lambda)) \succ 0$ for every $\lambda \in [0, 1]$.

This assumption ensures that the iterate tracks a continuous family of strict local minima rather than relying on global convexity. Near $\lambda = 0$, this condition is naturally satisfied. Because the initial state θ_0^* is a non-degenerate local minimum of L_{replay} , it follows that $H_0 = H_{\text{replay}}(\theta_0^*) \succ 0$. Consequently, the implicit function theorem guarantees a unique smooth branch in the immediate neighbourhood of $\lambda = 0$.

The persistence of this positive-definite branch as λ increases is bounded by Weyl's inequality applied to $H_\lambda = H_{\text{replay}} + \lambda H_{\text{current}}$:

$$\lambda_{\min}(H_\lambda) \geq \lambda_{\min}(H_{\text{replay}}) + \lambda \lambda_{\min}(H_{\text{current}}).$$

If the new task is locally convex at these points ($H_{\text{current}} \succeq 0$), positive-definiteness holds for all $\lambda \geq 0$ without requiring a global convexity assumption.

Part A: The Perfect Path Never Spikes

Let $\Theta^*(\lambda)$ be the ‘‘Optimum Path’’, the exact parameter state that perfectly minimises $L_\lambda(\theta)$ for any given λ .

Step 1: The First-Order Condition. Because $\Theta^*(\lambda)$ is the perfect minimum, the gradient (slope) of the total loss evaluated at that exact point must be zero. We define this as our starting equilibrium:

$$\begin{aligned} \nabla [L_\lambda(\Theta^*(\lambda))] &= 0 \\ \nabla [L_{\text{replay}}(\Theta^*(\lambda)) + \lambda \cdot L_{\text{current}}(\Theta^*(\lambda))] &= 0 \end{aligned}$$

Distributing the gradient operator to both terms gives us the fundamental First-Order Condition:

$$\nabla L_{\text{replay}}(\Theta^*(\lambda)) + \lambda \cdot \nabla L_{\text{current}}(\Theta^*(\lambda)) = 0 \quad (\text{D.2})$$

Step 2: Implicit Differentiation to Find Path Movement. We want to know how the optimal path Θ^* moves as λ changes. We apply the derivative operator $\frac{d}{d\lambda}$ to the entire First-Order Condition:

$$\frac{d}{d\lambda} [\nabla L_{\text{replay}}(\Theta^*(\lambda)) + \lambda \cdot \nabla L_{\text{current}}(\Theta^*(\lambda))] = \frac{d}{d\lambda} [0]$$

Distributing the operator to each term:

$$\frac{d}{d\lambda} [\nabla L_{\text{replay}}(\Theta^*(\lambda))] + \frac{d}{d\lambda} [\lambda \cdot \nabla L_{\text{current}}(\Theta^*(\lambda))] = 0$$

Evaluating the first term via the multivariable chain rule:

$$\frac{d}{d\lambda} [\nabla L_{\text{replay}}(\Theta^*(\lambda))] = \nabla^2 L_{\text{replay}}(\Theta^*(\lambda)) \cdot \frac{d\Theta^*}{d\lambda}$$

Evaluating the second term via the product rule and chain rule:

$$\frac{d}{d\lambda} [\lambda \cdot \nabla L_{\text{current}}(\Theta^*(\lambda))] = (1) \cdot \nabla L_{\text{current}}(\Theta^*(\lambda)) + \lambda \cdot \left(\nabla^2 L_{\text{current}}(\Theta^*(\lambda)) \cdot \frac{d\Theta^*}{d\lambda} \right)$$

Substituting these expanded terms back into the equation:

$$\left[\nabla^2 L_{\text{replay}}(\Theta^*) \cdot \frac{d\Theta^*}{d\lambda} \right] + \left[\nabla L_{\text{current}}(\Theta^*) + \lambda \cdot \nabla^2 L_{\text{current}}(\Theta^*) \cdot \frac{d\Theta^*}{d\lambda} \right] = 0$$

Grouping the terms that share the path movement $\frac{d\Theta^*}{d\lambda}$:

$$(\nabla^2 L_{\text{replay}}(\Theta^*) + \lambda \cdot \nabla^2 L_{\text{current}}(\Theta^*)) \cdot \frac{d\Theta^*}{d\lambda} + \nabla L_{\text{current}}(\Theta^*) = 0$$

Recognising the definition of the combined Hessian H_λ inside the parentheses:

$$H_\lambda \cdot \frac{d\Theta^*}{d\lambda} + \nabla L_{\text{current}}(\Theta^*(\lambda)) = 0$$

Solving algebraically for the path movement $\frac{d\Theta^*}{d\lambda}$:

$$\frac{d\Theta^*}{d\lambda} = -H_\lambda^{-1} \cdot \nabla L_{\text{current}}(\Theta^*(\lambda)) \quad (\text{D.3})$$

Step 3: Calculating the Change in Task 0 Loss. How does the Replay loss specifically change as λ increases along this perfect path? We apply the derivative operator:

$$\frac{d}{d\lambda} [L_{\text{replay}}(\Theta^*(\lambda))] = \nabla L_{\text{replay}}(\Theta^*(\lambda))^\top \cdot \frac{d\Theta^*(\lambda)}{d\lambda}$$

We perform two substitutions here. First, from (D.2), we know $\nabla L_{\text{replay}} = -\lambda \cdot \nabla L_{\text{current}}$. Second, from (D.3), we substitute the path movement.

$$\begin{aligned} \frac{d}{d\lambda} L_{\text{replay}}(\Theta^*(\lambda)) &= (-\lambda \cdot \nabla L_{\text{current}}(\Theta^*(\lambda)))^\top \cdot (-H_\lambda^{-1} \cdot \nabla L_{\text{current}}(\Theta^*(\lambda))) \\ &= \lambda \cdot \nabla L_{\text{current}}(\Theta^*(\lambda))^\top \cdot H_\lambda^{-1} \cdot \nabla L_{\text{current}}(\Theta^*(\lambda)) \end{aligned}$$

Because H_λ is strictly positive-definite along the followed branch of local minima for all $\lambda \in [0, 1]$ (per the Local-Minimum Assumption), its inverse H_λ^{-1} is also positive-definite. The quadratic form $v^\top M v$ is therefore guaranteed to be non-negative. Since $\lambda \geq 0$, the entire equation satisfies:

$$\frac{d}{d\lambda} L_{\text{replay}}(\Theta^*(\lambda)) \geq 0$$

Conclusion for Part A: The Replay loss mathematically never decreases along the perfect path, meaning it can never overshoot its final joint-optimum value.

Part B: The Clumsy Network Tracks the Perfect Path

Let $\theta(t)$ be the actual position of the model parameters at time t . We define the tracking error $\delta(t)$:

$$\delta(t) = \theta(t) - \Theta^*(\lambda(t))$$

Step 1: Differentiating the Error. We apply the time derivative operator $\frac{d}{dt}$ to our error definition:

$$\frac{d}{dt} [\delta(t)] = \frac{d}{dt} [\theta(t)] - \frac{d}{dt} [\Theta^*(\lambda(t))]$$

The model moves via gradient descent, so $\frac{d\theta}{dt} = -\nabla L_{\lambda(t)}(\theta(t))$. The perfect path moves via the chain rule $\frac{d\Theta^*}{dt} = \frac{d\Theta^*}{d\lambda} \cdot \frac{d\lambda}{dt}$. Because the curriculum has a constant velocity over N steps, $\frac{d\lambda}{dt} = \frac{1}{N}$.

$$\frac{d\delta}{dt} = -\nabla L_{\lambda(t)}(\theta(t)) - \left(\frac{d\Theta^*}{d\lambda} \cdot \frac{1}{N} \right)$$

Step 2: Taylor Expansion of the Actual Gradient. We approximate the gradient at the actual position $\theta(t)$ by Taylor-expanding it around the perfect position Θ^* :

$$\nabla L_\lambda(\theta) \approx \nabla L_\lambda(\Theta^*) + \nabla [\nabla L_\lambda(\Theta^*)] \cdot (\theta - \Theta^*)$$

Since the gradient at the perfect minimum is zero ($\nabla L_\lambda(\Theta^*) = 0$) and the derivative of the gradient is the Hessian ($\nabla^2 L_\lambda = H_\lambda$), this collapses to:

$$\nabla L_\lambda(\theta) \approx H_\lambda \cdot \delta$$

Step 3: The Final Steady State. Substitute this expanded gradient back into our differential equation:

$$\frac{d\delta}{dt} \approx -H_\lambda \cdot \delta - \frac{d\Theta^*}{d\lambda} \cdot \frac{1}{N}$$

Assuming the lag reaches a steady state, the error stops growing ($\frac{d\delta}{dt} \approx 0$). This is a first-order, gradient-flow approximation: it linearises the gradient around Θ^* and treats the dynamics as continuous, so it captures the scaling of the lag with N rather than an exact discrete-time bound.

$$\begin{aligned} 0 &\approx -H_\lambda \cdot \delta - \frac{d\Theta^*}{d\lambda} \cdot \frac{1}{N} \\ H_\lambda \cdot \delta &\approx -\frac{d\Theta^*}{d\lambda} \cdot \frac{1}{N} \\ \delta &\approx -H_\lambda^{-1} \cdot \frac{d\Theta^*}{d\lambda} \cdot \frac{1}{N} \end{aligned}$$

Taking the norm, we see the lag distance is mathematically bounded by the curriculum length N :

$$\|\delta\| = O\left(\frac{1}{N}\right) \tag{D.4}$$

Combining Part A and B: Expanding the Actual Loss

To find the maximum Replay loss the model will actually experience, we Taylor expand the loss of the actual trajectory around the perfect position:

$$\begin{aligned} L_{\text{replay}}(\theta) &\approx L_{\text{replay}}(\Theta^*) + \nabla L_{\text{replay}}(\Theta^*)^\top \cdot (\theta - \Theta^*) \\ L_{\text{replay}}(\theta) &\approx L_{\text{replay}}(\Theta^*) + \nabla L_{\text{replay}}(\Theta^*)^\top \cdot \delta \end{aligned}$$

From Part A, $L_{\text{replay}}(\Theta^*)$ cannot exceed the final joint optimum loss $L_{\text{replay}}(\theta_{\text{joint}}^*)$. From Part B, the lag δ is $O(1/N)$. Substituting these limits yields the final bound:

$$\max_t L_{\text{replay}}(\theta(t)) \leq L_{\text{replay}}(\theta_{\text{joint}}^*) + O\left(\frac{1}{N}\right) \tag{D.5}$$

Conclusion: The transient stability gap is entirely contained within the $O(1/N)$ term, meaning a sufficiently slow continuous curriculum eliminates the discrete-time overshoot.